

Java 高级开发面试题完整答案

从基础原理到生产实践的系统化解析

面试复习讲义

2026-06

阅读说明

本讲义面向具有 5 年以上经验、目标岗位为高级 Java 开发或技术骨干的读者。它不是一份只给关键词的“八股答案”，而是按照真实面试的表达顺序组织：先给出一句话结论，再解释底层原理，随后落到代码、线上场景和方案取舍。

版本语境

- Java 集合部分以 JDK 8 之后的主流实现为主，同时说明 JDK 7 的关键差异。
- JVM 与诊断部分以现代 HotSpot、JDK 17/21 的生产实践为主；传统命令仍保留，因为大量存量系统仍在使用。
- Spring 部分以 Spring Framework 6 的代理和事务语义为准，并区分“框架公开语义”与“源码实现细节”。
- 数据库部分以 MySQL 8/InnoDB 为主要语境。
- Redis、消息队列与分布式系统部分重点讨论可靠性边界，不把任何中间件能力描述成无条件的“绝对一致”。

推荐的面试表达结构

面对原理题，可以用四步回答：

1. **先下结论**：用 20 秒给出核心判断。
2. **解释机制**：说明数据结构、状态转换或 happens-before 关系。
3. **补充边界**：指出版本差异、异常路径和不成立的条件。
4. **落到实践**：说明生产环境如何选型、监控和止损。

面对系统设计题，则建议按照“目标与约束 → 流量路径 → 数据一致性 → 故障处理 → 可观测性 → 取舍”展开。

目录

1. HashMap 为什么线程不安全
2. volatile 能否保证线程安全
3. synchronized 与 ReentrantLock 如何选择
4. 线程池参数如何配置
5. Java 服务 CPU 飙升如何排查
6. 内存泄漏与 OOM 如何排查
7. Spring 如何处理循环依赖
8. Spring 事务为什么会失效
9. 数据库索引为什么会失效
10. 如何保证接口幂等性
11. Redis 缓存与数据库如何保持一致
12. 消息队列如何保证不丢消息
13. Redis 分布式锁应该如何实现
14. 高并发秒杀系统设计
15. 编程题：线程安全 LRU 缓存
16. 接口变慢但 CPU、内存正常如何排查
17. 综合项目题的高质量回答方法
18. 附录：命令、公式与检查清单

1. HashMap 为什么线程不安全？

1.1 结论先行

HashMap 的设计目标是单线程下提供接近 $O(1)$ 的查询与更新，它没有为复合操作、结构修改和内存可见性提供并发保证。多个线程同时 put、扩容或遍历时，可能出现数据覆盖、更新丢失、读取到中间状态、遍历异常等问题。JDK 7 在并发扩容的特定交错下还可能形成链表环，导致查询死循环；JDK 8 改变了扩容迁移方式，显著降低了该类风险，但 HashMap 仍然不是线程安全容器。

面试中不要只回答“因为没有加锁”。更准确的说法是：**它的内部不变量需要由一系列非原子步骤共同维护，而这些步骤既没有互斥，也没有建立完整的可见性与有序性关系。**

1.2 JDK 7 与 JDK 8 的数据结构差异

对比项	JDK 7	JDK 8 及以后主流实现
桶内结构	数组 + 单向链表	数组 + 单向链表 + 红黑树
新节点插入	通常为头插法	通常为尾插法
扩容迁移	逐节点重新计算并头插到新桶	利用容量为 2 的幂，将节点拆分到原索引或原索引 + oldCap
链表树化	不支持	桶内节点达到阈值且数组容量满足条件时树化
极端碰撞复杂度	$O(n)$	树化后理想情况下 $O(\log n)$

JDK 8 的树化阈值通常是 8，反树化阈值通常是 6，且表容量小于 64 时优先扩容而不是树化。面试时应强调这些是实现细节，不能据此编写依赖内部阈值的业务逻辑。

1.3 put 为什么不是一个原子动作

一次 put 至少可能包含以下步骤：

1. 计算 key 的哈希扰动值。
2. 根据当前数组长度计算桶下标。
3. 读取桶头，遍历链表或红黑树。
4. 判断是覆盖旧值还是插入新节点。
5. 修改桶链接关系或树结构。
6. 更新 size、modCount。
7. 判断是否超过阈值并触发扩容。

两个线程可能同时观察到同一个旧状态。例如，它们都认为某个桶为空，然后分别写入自己的节点，后写入者会覆盖先写入者，表现为更新丢失；也可能同时修改同一条链的 next 指针，使内部结构处于不一致状态。

即使多个线程只做 get，只要另外存在没有安全发布的写线程，也不能依据 Java 内存模型断言读线程一定看见最新且完整的结构。

1.4 JDK 7 并发扩容为何可能形成环

JDK 7 扩容时会把旧链表节点迁移到新数组，并采用头插法反转链表顺序。假设旧桶是：

```
A -> B -> null
```

线程 T1 读取到 A.next = B 后被挂起；线程 T2 完成扩容，把链表迁移并反转：

```
B -> A -> null
```

T1 恢复后仍持有旧的局部引用，并按照旧的迁移逻辑继续修改 next。在特定交错下，可能得到：

```
A -> B -> A -> ...
```

之后对该桶执行查找会无限遍历，CPU 可能被打满。

这里的关键不是背诵某一张执行时序图，而是理解：两个线程在没有互斥的情况下共同重写同一条链的指针，并且迁移算法会改变链表方向，因此可能破坏“链最终以 null 结束”的结构不变量。

1.5 JDK 8 是否就安全了

不是。JDK 8 的迁移算法不再以相同方式反转链表，避免了 JDK 7 最著名的链表环问题，但仍可能出现：

- 同时插入导致覆盖或更新丢失。
- size 统计不准确。
- 遍历期间发生结构修改，触发 ConcurrentModificationException，或得到弱一致之外的非预期结果。
- 业务层“先判断再写入”产生竞态：

```
if (!map.containsKey(key)) {
    map.put(key, value);
}
```

即便把 map 替换成线程安全容器，这段复合逻辑也未必满足业务原子性，应使用 putIfAbsent、computeIfAbsent 或外部同步。

1.6 ConcurrentHashMap 如何解决

JDK 7：分段锁

JDK 7 的 ConcurrentHashMap 使用 Segment 分段，每个 Segment 类似一个可重入锁保护的小型哈希表。不同分段的写入可以并发，但并发度受分段数量限制。

JDK 8：CAS + 桶级同步

JDK 8 移除了固定 Segment 数组，核心思想是：

- 空桶首次写入通常通过 CAS 安装节点。
- 桶非空时，对桶头节点使用 `synchronized`，把锁粒度控制在单个桶。
- 扩容时使用转发节点标识迁移状态，多个线程可以协助迁移。
- 计数采用类似分散热点的方式，降低单一计数器竞争。
- 读取多数情况下不加锁，节点关键字段通过内存语义保证可见性。

注意：ConcurrentHashMap 只保证它提供的单个操作或指定复合方法的线程安全，不自动保证跨多个 key、跨多个容器或“读-改-写”业务流程的原子性。

1.7 生产环境如何选择

- 单线程、方法局部变量、构建完成后安全发布且只读：可以使用 HashMap。
- 多线程高并发读写：优先考虑 ConcurrentHashMap。
- 需要保持插入顺序或访问顺序：应结合锁使用 LinkedHashMap，或者重新设计数据结构。
- 需要不可变快照：构建后发布不可变 Map，通常比共享可变容器更容易推理。
- 需要跨 key 的事务性约束：单靠并发 Map 不够，应使用锁、数据库约束、状态机或串行化执行。

1.8 常见误区与追问

误区一：“HashMap 读操作没有修改，所以并发读写安全。”

错误。读取依赖写线程正确发布的数据结构；并发写也可能让读线程观察到暂态结构。

误区二：“给 HashMap 引用加 volatile 就安全了。”

错误。volatile 只能保证引用本身的可见性，不能让容器内部多个字段更新变成原子事务。

追问：ConcurrentHashMap 的迭代器是否强一致？

通常是弱一致：不会像普通快速失败迭代器那样依赖全局锁，可以反映创建迭代器时或之后的部分修改，但不承诺某个瞬间的完整快照。

追问：size() 能否作为并发控制条件？

不应。并发状态随时变化，`if (map.size() < limit) map.put(...)` 不是原子操作。配额控制应使用原子计数、信号量或外部一致性机制。

2. volatile 能保证线程安全吗？

2.1 结论先行

volatile 不能笼统地“保证线程安全”。它主要提供：

- 对该变量写入后，其他线程后续读取能够看到更新。
- 对 volatile 写之前的普通读写，不会被重排到 volatile 写之后；对 volatile 读之后的普通读写，不会被重排到 volatile 读之前。
- 对单次 volatile 读写本身提供原子性，但不保证由多步组成的复合操作原子性。

因此下面的代码不是线程安全计数器：

```
public class Counter {  
    private volatile int count = 0;  
  
    public void increment() {  
        count++;  
    }  
}
```

count++ 本质上至少是“读取旧值 → 加一 → 写回”三个步骤。两个线程都可能读取到 10，然后都写回 11，丢失一次更新。

2.2 volatile 的三类语义

可见性

线程 A 对 volatile 变量的写，happens-before 线程 B 后续对同一个 volatile 变量的读。借助这条规则，A 在写 volatile 之前完成的普通写入，也能被随后读到该 volatile 值的 B 看见。

典型用途是停止标志：

```
public final class Worker implements Runnable {  
    private volatile boolean stopped;  
  
    public void stop() {  
        stopped = true;  
    }  
  
    @Override  
    public void run() {  
        while (!stopped) {  
            doOneUnit();  
        }  
    }  
}
```

前提是循环体不会永久阻塞，并且停止动作不要求与其他复合状态原子切换。

有序性

JIT 和 CPU 可以在不改变单线程语义的前提下重排指令。volatile 通过内存屏障和平台相关指令约束某些重排，用于建立发布与订阅关系。

经典例子是双重检查锁定：

```
public final class Singleton {
    private static volatile Singleton instance;

    public static Singleton getInstance() {
        Singleton local = instance;
        if (local == null) {
            synchronized (Singleton.class) {
                local = instance;
                if (local == null) {
                    local = new Singleton();
                    instance = local;
                }
            }
        }
        return local;
    }
}
```

这里的 volatile 主要防止对象引用在构造完成之前被其他线程观察到，并提供可见性。更简单的单例实现通常是静态内部类或枚举。

单次读写原子性

对 volatile 变量的一次读或一次写不会被其他线程看到“半个值”。但 i++、check-then-act、多个字段的一致更新都不属于单次读写。

2.3 正确修改方式

方案一：AtomicInteger

```
public final class Counter {
    private final AtomicInteger count = new AtomicInteger();

    public int increment() {
        return count.incrementAndGet();
    }
}
```

其核心是 CAS：比较内存中的当前值是否仍等于预期值，如果是则写入新值，否则重试。CAS 避免了互斥阻塞，但高竞争下可能产生大量自旋。

方案二：LongAdder

当场景是高并发统计，且不要求每次读取都得到线性一致的瞬时值，可以使用 LongAdder：

```
private final LongAdder requests = new LongAdder();

public void record() {
```



```

    requests.increment();
}

public long total() {
    return requests.sum();
}

```

LongAdder 通过分散到多个 Cell 降低热点竞争，汇总时再求和。它适合指标计数，不适合生成唯一序号、账户余额或库存扣减等必须严格原子化的场景。

方案三：锁

当更新涉及多个变量、需要验证不变量或伴随外部操作时，应使用锁：

```

public synchronized void transfer(Account from, Account to, long amount) {
    if (from.balance() < amount) {
        throw new IllegalStateException("insufficient balance");
    }
    from.debit(amount);
    to.credit(amount);
}

```

实际转账还要考虑锁顺序、数据库事务和跨进程一致性。

2.4 内存屏障应如何回答

面试时不必把所有 CPU 指令背下来，可以用抽象语义回答：

- volatile 写具有 release 语义：它之前的普通写不能越过它跑到后面。
- volatile 读具有 acquire 语义：它之后的普通读写不能越过它跑到前面。
- JMM 把这些语义映射到不同硬件平台所需的屏障或带内存序的指令。

不要声称 Java 的每次 volatile 读写在所有 CPU 上都会生成完全相同的指令；实现会根据架构内存模型优化。

2.5 volatile 适用与不适用场景

适用：

- 状态标志、配置快照引用。
- 单写多读，且写入之间不存在复合不变量。
- 安全发布不可变对象。
- 配合锁实现快速路径，例如双重检查。

不适用：

- 计数器自增。
- 余额、库存等读改写。

- 多个字段必须同时变化。
- 需要“先判断再执行”的业务原子性。

2.6 进一步追问

volatile 与 synchronized 的关系是什么？

二者都能建立可见性和有序性，但 synchronized 还提供互斥，使临界区内的复合操作对其他线程表现为串行执行。

CAS 有什么问题？

高竞争自旋、ABA、只能原子操作有限状态。ABA 可通过版本号或 AtomicStampedReference 缓解；复杂不变量通常直接用锁更清晰。

为什么 LongAdder.sum() 不适合强一致余额？

汇总期间其他线程仍可能更新不同 Cell，结果是统计意义上的近实时值，而不是全局线性化点上的余额。

3. synchronized 和 ReentrantLock 如何选择?

3.1 结论先行

默认优先选择 synchronized：语义简单、异常时自动释放、JVM 能进行锁消除和锁粗化等优化，代码更不容易出错。当业务明确需要可中断获取、超时尝试、公平策略、多个条件队列或更细粒度的锁状态查询时，再使用 ReentrantLock。

不要用“ReentrantLock 性能一定更高”作为选型理由。现代 JVM 中，两者性能取决于竞争模式、临界区长度和运行环境，正确性与可维护性通常比微基准更重要。

3.2 核心能力对比

能力	synchronized	ReentrantLock
获取/释放	进入、退出监视器；异常自动释放	显式 lock()/unlock()，必须 finally
可重入	支持	支持
可中断等待	普通进入不可直接响应中断	lockInterruptibly()
超时尝试	不直接支持	tryLock(timeout, unit)
公平锁	不提供公平性配置	构造时可指定公平策略
条件队列	每个对象监视器一组 wait-set	可创建多个 Condition
状态查询	能力有限	可查询持有、排队等状态
JVM 优化	与语言/JVM 深度集成	基于 AQS，仍有成熟优化

3.3 正确使用模板

```
private final ReentrantLock lock = new ReentrantLock();

public void update() {
    lock.lock();
    try {
        // 维护共享状态不变量
    } finally {
        lock.unlock();
    }
}
```

必须把 unlock() 放在成功获取锁之后对应的 finally 中。不要把可能抛异常的代码放在 lock() 和 try 之间。

3.4 AQS 的理解框架

ReentrantLock 基于 AbstractQueuedSynchronizer。可以从三个概念理解：

1. 一个原子状态字段表示同步状态，例如独占锁的重入次数。

2. 获取失败的线程进入 FIFO 风格的等待队列。
3. 释放锁时唤醒合适的后继节点，由其再次竞争。

AQS 提供同步框架，但具体的“状态代表什么、获取和释放条件是什么”由子类实现。Semaphore、CountDownLatch、ReentrantReadWriteLock 等也复用这一思想。

3.5 公平锁是否更好

公平锁倾向于让等待时间更久的线程先获得锁，可以降低饥饿风险，但通常牺牲吞吐量：刚释放锁的线程或新到线程不能充分利用局部性和调度时机，需要更多上下文切换。

选择公平锁的典型原因是业务对等待上界或顺序有明确要求，而不是为了“更公平看起来更正确”。多数在线服务更关注总体吞吐和尾延迟，默认非公平锁通常更合适。

3.6 Condition 的价值

synchronized 下一个对象只有一组 wait/notify 等待集合；ReentrantLock 可以建立多个条件队列，将不同原因的等待者分开。

```
private final ReentrantLock lock = new ReentrantLock();
private final Condition notEmpty = lock.newCondition();
private final Condition notFull = lock.newCondition();
```

有界队列中，生产者只需唤醒等待“非满”的线程，消费者只需唤醒等待“非空”的线程，避免无差别唤醒。

等待条件必须使用 while 而不是 if:

```
while (queue.isEmpty()) {
    notEmpty.await();
}
```

原因包括虚假唤醒，以及线程被唤醒后条件可能已被其他线程再次改变。

3.7 一个必须使用 ReentrantLock 的场景

订单聚合服务需要同时获取两个资源锁，但不能无限等待，否则会拖垮请求线程；同时系统希望在服务关闭时中断等待线程：

```
boolean first = false;
boolean second = false;
try {
    first = lockA.tryLock(100, TimeUnit.MILLISECONDS);
    if (!first) {
        throw new BusyException();
    }
    second = lockB.tryLock(100, TimeUnit.MILLISECONDS);
    if (!second) {
        throw new BusyException();
    }
    merge();
} finally {
    if (second) lockB.unlock();
```

```
    if (first) lockA.unlock();  
}
```

synchronized 无法直接提供带超时的获取语义。更进一步，应通过固定锁顺序或更高层的数据分区方案减少双锁需求。

3.8 读写锁与 StampedLock

读多写少时，ReentrantReadWriteLock 可能提高并发度，但前提是读临界区足够长、竞争明显；否则锁管理成本可能抵消收益。写锁降级为读锁可以设计，但读锁升级为写锁容易死锁，应释放读锁后重新竞争写锁并再次校验条件。

StampedLock 支持乐观读，但不可重入、API 更易误用。乐观读必须在读取后调用 validate，失败则退化为悲观读。它适合性能敏感的局部数据结构，不应在普通业务代码中为了“高级”而使用。

3.9 常见误区

- 只要用了锁就线程安全：错误。锁对象必须一致，且所有访问共享状态的路径都遵守同一协议。
- tryLock() 失败后仍执行临界区：错误。
- 临界区内调用不受控的远程接口：容易长时间持锁，应先设计状态转移和补偿。
- 锁住字符串常量或可被外部访问的对象：可能与未知代码形成意外锁竞争。

4. 线程池参数应该如何配置？

4.1 题目数据与第一版估算

已知：

- 单次请求平均耗时 100 ms。
- 其中 CPU 计算 20 ms, I/O 等待 80 ms。
- 8 核 CPU。
- 峰值 QPS 500。

有两个常用估算角度。

角度一：Little 定律估算并发数

稳定状态下：

$$\begin{aligned}\text{系统内平均并发任务数} &\approx \text{吞吐量} \times \text{平均响应时间} \\ &\approx 500 \times 0.1 \\ &\approx 50\end{aligned}$$

这意味着，要在没有大量排队的理想条件下承接 500 QPS，至少需要大约 50 个同时在途的执行槽位。

角度二：CPU/等待比公式

$$\begin{aligned}\text{线程数} &\approx \text{CPU 核数} \times \text{目标利用率} \times (1 + \text{等待时间} / \text{计算时间}) \\ &\approx 8 \times 0.8 \sim 1.0 \times (1 + 80 / 20) \\ &\approx 32 \sim 40\end{aligned}$$

两个结果不完全一致并不矛盾：公式依赖平均值和理想假设。50 是从端到端吞吐需求推导的在途并发；32~40 是从 CPU 利用和等待比推导的执行线程建议。实际配置必须结合连接池容量、下游限额、尾延迟和压测结果。

一个可用于首轮压测的配置可能是：

```
int core = 40;
int max = 50;
int queueCapacity = 500; // 约 1 秒峰值流量，必须结合 SLA 调整

ThreadPoolExecutor executor = new ThreadPoolExecutor(
    core,
    max,
    60,
    TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(queueCapacity),
    new NamedThreadFactory("order-query"),
    new ThreadPoolExecutor.AbortPolicy()
);
```

这不是“标准答案”，而是可验证的初始假设。最终数值必须通过压测与线上指标校准。

4.2 为什么不能只按 CPU 核数配置

I/O 密集型任务大量时间处于等待状态，线程数可以高于 CPU 核数，以便一个线程等待时其他线程利用 CPU。但线程也不是越多越好：

- 每个线程占用栈空间和原生资源。
- 上下文切换增加。
- 下游数据库、Redis、HTTP 连接池可能先被耗尽。
- 更多并发会把压力传给本已变慢的依赖，形成拥塞放大。

线程池配置本质上是**系统容量与背压策略**，而不是一道纯数学题。

4.3 corePoolSize 与 maximumPoolSize 如何协同

ThreadPoolExecutor 默认执行路径是：

1. 当前线程数小于 core，创建核心线程。
2. 否则优先把任务放入队列。
3. 队列满后，若线程数小于 max，再创建非核心线程。
4. 线程数达到 max 且队列也满，执行拒绝策略。

因此使用很大的无界队列时，maximumPoolSize 几乎不会发挥作用，任务会持续堆积，最终表现为延迟上升或 OOM。

4.4 队列为何应优先有界

有界队列把系统容量显式化，允许在过载时尽早失败、降级或回退。无界队列看似“不拒绝任务”，实际只是把拒绝延后成更危险的内存耗尽和超长尾延迟。

队列容量可从允许的排队时间估算：

队列容量 $\approx$ 可接受排队时长 $\times$ 可持续消费速率

例如只允许额外排队 200 ms，消费能力约 500 QPS，则队列容量约 100。生产配置还应留出突发余量，但不能把队列当作无限缓冲区。

4.5 队列类型如何选

- ArrayBlockingQueue：有界、内存连续、容量明确，适合大多数业务线程池。
- LinkedBlockingQueue：可有界；如果不传容量，容量极大，生产环境要谨慎。
- SynchronousQueue：不存储任务，直接移交给工作线程，适合希望快速扩线程且任务短的场景。
- PriorityBlockingQueue：无界优先队列，需防低优先级饥饿和内存增长。

- DelayQueue: 延迟任务专用, 不应拿来做普通请求池。

4.6 拒绝策略如何选择

- AbortPolicy: 直接抛异常, 适合调用方能够感知失败并快速降级的在线请求。
- CallerRunsPolicy: 由提交任务的线程执行, 形成自然背压; 但如果提交线程是 Web I/O 线程, 会增加请求延迟, 必须评估。
- DiscardPolicy: 静默丢弃, 除非任务天然可丢且有指标, 否则风险很高。
- DiscardOldestPolicy: 丢弃队头任务再重试, 可能破坏顺序和业务语义。
- 自定义策略: 记录指标、采样日志、限流、写入降级队列或返回明确错误。

面试高分答案应说明: **拒绝不是异常边角, 而是线程池容量设计的一部分。**

4.7 线程池与下游连接池的约束

假设数据库连接池只有 30 个连接, 而线程池开到 100。大量线程会阻塞在取连接上, 占据线程和队列, 导致超时扩散。通常要满足:

- 并发访问数据库的线程数不应无视连接池上限。
- 多个接口共享连接池时, 要为关键路径预留容量。
- 对不同依赖和不同 SLA 的任务做线程池隔离, 避免一个慢依赖拖死全站。

这就是舱壁模式: 订单查询、报表导出、异步通知等不应随意共用同一个大池。

4.8 监控指标

至少监控:

- poolSize、activeCount、largestPoolSize。
- queue.size、队列容量使用率。
- taskCount、completedTaskCount。
- 拒绝次数与拒绝原因。
- 任务排队时间、执行时间、总耗时的 P50/P95/P99。
- 线程状态分布、异常数。
- 下游连接池等待时间、超时率。

只看活跃线程数不够。一个线程池可能活跃度不高, 但队列中积压的是超时后已无业务价值的任务。

4.9 为什么不建议直接使用 Executors 工厂

问题不在于 API 本身“绝对不能用”，而在于一些便捷方法隐藏了关键容量选择：

- `newFixedThreadPool` 常配合近似无界队列，可能持续积压。
- `newCachedThreadPool` 的最大线程数非常大，可能创建过多线程。
- `newSingleThreadExecutor` 同样可能无界积压。
- 默认线程工厂命名和异常处理不利于定位。

生产环境更推荐显式构造 `ThreadPoolExecutor` 或使用经过治理的平台封装，明确线程数、队列、拒绝、命名、监控和上下文传播。

4.10 进一步提高答案质量

给出配置后还应主动补充：

1. 平均耗时不足以描述尾延迟，应基于 P95/P99 和依赖超时进行压测。
2. 500 QPS 是入口峰值还是分配到单实例后的峰值，需要区分。
3. 单实例 8 核是否有容器 CPU quota，JVM 看到的核数是否正确。
4. 线程池任务是否包含同步远程调用，是否能改为异步 I/O。
5. 过载时是排队、拒绝、降级还是扩容，应由 SLA 决定。

5. 如何排查 Java 服务 CPU 飙升？

5.1 结论先行：先止损，再保留证据，最后定位

CPU 飙升排查不能只执行一次 jstack 就下结论。一个可靠流程是：

1. 确认是整机、容器还是单个 Java 进程占用 CPU。
2. 判断是用户态计算、系统调用、I/O wait 还是虚拟化偷取时间。
3. 定位 Java 进程内的高 CPU 线程。
4. 将操作系统线程 ID 转为十六进制，与线程栈中的 nid 对应。
5. 连续采集多次线程栈，确认热点栈是否稳定。
6. 结合 JFR/async-profiler 火焰图、GC 日志和业务指标验证。
7. 根据原因执行限流、降级、扩容、回滚或代码修复。

线上排查的第一原则是：**不要因为诊断动作造成二次故障**。大堆、极高负载时生成 Heap Dump 可能带来长时间停顿或大量磁盘写入；应先评估风险。

5.2 第一步：判断 CPU 到底消耗在哪里

```
uptime
top
mpstat -P ALL 1
vmstat 1
pidstat -u -p <pid> 1
```

关注：

- %us 高：用户态代码计算密集，可能是死循环、序列化、正则、加解密、压缩、业务热点。
- %sy 高：系统调用频繁，可能是网络、文件、锁竞争、线程调度或内核问题。
- %wa 高：CPU 本身未必忙，线程在等待磁盘等 I/O。
- load average 高但 CPU 不高：可能大量线程处于不可中断 I/O 等待或运行队列拥塞。
- 容器环境要同时看 cgroup CPU quota 和 throttling，宿主机指标不能直接代表容器可用算力。

先确认 PID：

```
jps -lv
ps -ef | grep java
```

现代环境也可以使用：

```
jcmd -l
```

5.3 第二步：定位进程内高 CPU 线程

```
top -Hp <java_pid>
```

或者：

```
pidstat -t -p <java_pid> 1
```

记录持续占用 CPU 的线程 ID，例如十进制 12345。然后转换为十六进制：

```
printf '%x\n' 12345
```

假设输出 3039，在线程栈中搜索：

```
jstack <java_pid> > /tmp/jstack-1.txt
grep -n -A 30 -B 5 'nid=0x3039' /tmp/jstack-1.txt
```

在现代 JDK 上，官方更推荐使用 jcmd：

```
jcmd <java_pid> Thread.print -l > /tmp/threads-1.txt
```

应间隔 3~10 秒采集至少 3 次：

```
for i in 1 2 3; do
  date
  jcmd <java_pid> Thread.print -l > /tmp/threads-$i.txt
  sleep 5
done
```

如果同一线程持续停留在同一业务方法、同一循环或同一正则处理栈，证据比单次快照可靠得多。

5.4 如何区分常见原因

死循环或算法热点

典型特征：

- 某一或少数线程长期处于 RUNNABLE。
- 多次线程栈位置相近。
- GC 压力未必明显。
- 火焰图出现非常窄而高的热点栈。

常见代码：错误循环条件、递归无退出、哈希冲突、超大集合排序、灾难性正则回溯、JSON 深层解析、重复重试无退避。

频繁 GC

典型特征：

- GC 线程消耗大量 CPU。
- 应用吞吐下降，分配速率很高。
- GC 日志显示 Young GC 极频繁，或 Full GC 周期性发生且回收效果差。

- 堆使用呈锯齿但基线不断抬升，可能存在泄漏；若每次都能回落，可能只是分配过快或堆配置偏小。

建议启用统一 GC 日志，例如：

```
-Xlog:gc*,safepoint:file=/var/log/app/gc.log:time,uptime,level,tags:filecount=10,filesize=100M
```

关注暂停时间、GC 原因、回收前后占用、分配失败、晋升失败和并发周期是否跟不上。

锁竞争

锁竞争本身有时表现为线程阻塞而 CPU 不高；但高频短锁、CAS 自旋、上下文切换会消耗 CPU。特征包括：

- 大量线程栈聚集在同一锁或 AQS 获取路径。
- BLOCKED 线程很多，持锁线程执行远程调用或慢操作。
- JFR 中 Java Monitor Blocked、Thread Park、Lock Instances 等事件明显。
- 系统上下文切换次数升高。

序列化、日志或异常风暴

如果业务异常被高频抛出，构造堆栈、格式化日志、同步写磁盘都可能消耗 CPU。应检查：

- 单位时间异常数。
- 相同错误日志是否刷屏。
- 日志队列是否满、是否退化为同步。
- 是否在热点循环中调用 String.format、复杂 JSON 序列化或打印大对象。

5.5 火焰图与 JFR

线程栈是离散快照，火焰图更适合回答“CPU 时间主要花在哪条调用链”。推荐生产诊断工具：

- Java Flight Recorder: JDK 自带、适合持续或短时低开销采样。
- async-profiler: 支持 CPU、分配、锁等事件，能减少 safepoint bias。

JFR 示例：

```
jcmd <pid> JFR.start name=cpu settings=profile duration=60s filename=/tmp/cpu.jfr
```

然后用 JDK Mission Control 查看热点方法、线程、锁、分配与 GC。

火焰图阅读要点：

- 横向宽度代表采样占比，不代表时间顺序。
- 顶部宽的叶子方法通常是直接热点。
- 底部宽表示大量热点共享的调用入口。
- 应区分业务代码、JVM 运行时、GC、内核和 native 库。

5.6 线上止损策略

- 单接口异常：网关限流、熔断、关闭高成本功能、缩短超时。
- 新版本引入：快速回滚或灰度摘除。
- 单实例热点：从负载均衡摘除，保留一台用于诊断。
- 重试风暴：限制重试次数，增加指数退避与随机抖动。
- 正则/输入攻击：限制请求体、字段长度和处理时间。
- GC 压力：临时扩容或降低流量，但根因仍需修复对象分配或泄漏。

不要直接用 kill -9 作为第一动作。优先保留线程栈、JFR、GC 日志、应用日志、版本与流量信息；但若实例已影响整个集群，业务止损优先级高于证据完整性。

5.7 一份可直接复述的回答

我会先确认 CPU 是整机、容器还是单个 JVM 的问题，并区分 user、system、iowait。然后用 `top -Hp` 或 `pidstat -t` 找持续高 CPU 的线程，把十进制 TID 转成十六进制，在连续三份 `jcmd Thread.print` 中匹配 `nid`。如果栈稳定在业务循环，继续用 JFR 或 `async-profiler` 看 CPU 火焰图；如果热点是 GC 线程，就结合 GC 日志、分配速率和堆曲线判断是堆太小、分配过快还是泄漏；如果大量线程聚集在锁获取，则检查持锁者和临界区。排查过程中先做限流、回滚、摘除等止损，并保留现场证据。

6. 如何排查内存泄漏和 OOM?

6.1 先区分“内存泄漏”与“内存不足”

- **内存泄漏**：对象已无业务价值，但仍被 GC Root 可达，因此无法回收。其结果可能是 OOM，也可能只是长期占用上升。
- **内存不足**：业务确实需要的活动数据超过可用内存，例如一次加载 10 GB 文件到 2 GB 堆中，不一定存在泄漏。
- **非堆或原生资源耗尽**：Java 堆可能很健康，但直接内存、线程、类元数据、文件描述符或 native 库内存已经耗尽。

排查 OOM 的第一步是读取完整异常类型、发生线程、GC 日志与进程内存，而不是一看到 OOM 就盲目增大 -Xmx。

6.2 Java heap space

表示 JVM 无法在 Java 堆中为新对象分配足够空间，且 GC 后仍不能满足分配。

常见原因：

- 缓存、Map、队列没有上限。
- 监听器、回调、静态集合只增不减。
- 大批量查询一次性加载全部结果。
- 消息积压在应用内存而不是 Broker。
- 对象生命周期过长或引用未清理。
- 堆配置相对真实工作集过小。

推荐启动参数：

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/data/dumps
-XX:+ExitOnOutOfMemoryError
```

ExitOnOutOfMemoryError 是否使用要结合容器编排和故障恢复策略。某些 OOM 后进程状态不可预测，快速退出并由平台拉起通常比带病运行更安全，但必须避免重启风暴。

手动获取堆转储，现代 JDK 推荐：

```
jcmd <pid> GC.heap_dump filename=/data/dumps/heap.hprof
```

生成前确认磁盘空间和停顿风险。大堆场景可先采集类直方图：

```
jcmd <pid> GC.class_histogram
```

6.3 Metaspace

Metaspace 存储类元数据，使用本地内存而非传统 Java 堆。常见原因：

- 动态生成大量类，例如代理、脚本、字节码增强。
- 应用热部署后旧 ClassLoader 因线程、ThreadLocal、静态引用或驱动注册未释放。
- 大量不同表达式被编译为新类且缓存无界。
- MaxMetaspaceSize 配置过小。

排查重点：

- 类加载数量是否持续上升。
- 是否存在大量重复类名由不同 ClassLoader 加载。
- 哪个 ClassLoader 持有最多类，为什么不能卸载。
- Full GC 后类数量是否下降；类卸载是否开启并发生。

可使用：

```
jcmd <pid> VM.classloader_stats  
jcmd <pid> GC.class_histogram  
jcmd <pid> VM.native_memory summary
```

如果未在启动时启用 Native Memory Tracking，后续无法完整追溯历史；可在关键生产系统预先以 -XX:NativeMemoryTracking=summary 开启，注意其有一定开销。

6.4 Direct buffer memory

直接缓冲区通过堆外内存支持高效 I/O。常见问题：

- Netty/NIO Buffer 未正确释放或引用长期存活。
- 请求突发导致同时存在大量大 Buffer。
- Java 堆较大挤压了进程总内存，留给直接内存和线程栈的空间不足。
- 容器内存限制小于 JVM 对堆外资源的总体预期。

排查：

- 查看进程 RSS 与 Java 堆差异。
- 使用 NMT 分析 JVM 内部原生内存。
- 查看直接缓冲池指标，例如 BufferPoolMXBean。
- 对 Netty 开启适度的泄漏检测用于复现环境；最高级别检测开销较高，不宜长期全量开启。
- 检查 -XX:MaxDirectMemorySize、容器限制、堆、线程栈和 native 库的总预算。

不能只通过增大 MaxDirectMemorySize 解决。如果 Buffer 生命周期失控，增大上限只是延迟失败。

6.5 unable to create new native thread

JVM 创建 Java 线程时需要操作系统原生线程和栈空间。失败可能因为：

- 线程泄漏或线程池无上限。
- 每线程栈 -Xss 较大，虚拟内存或物理内存不足。
- 操作系统用户进程/线程数限制，例如 ulimit -u。
- 容器 PID limit。
- 进程地址空间被堆、直接内存、映射文件占满。

排查：

```
ps -elf | grep <pid> | wc -l
cat /proc/<pid>/status | grep Threads
ulimit -u
cat /sys/fs/cgroup/pids.max 2>/dev/null
jcmd <pid> Thread.print
```

重点找到线程创建源。常见错误是每个请求 new Thread()、连接失败后重复创建调度器、使用无界 newCachedThreadPool、线程池实例随对象反复创建却从不关闭。

6.6 GC Root 是什么

GC 通过可达性分析判断对象是否存活。GC Root 是遍历起点，典型包括：

- 活跃线程栈中的局部变量和参数。
- 已加载类的静态字段。
- JNI 全局引用和部分本地引用。
- JVM 内部结构、同步监视器等。

“某对象占用很大”不等于“它泄漏”。真正关键是：它为什么仍从 GC Root 可达，以及这条引用链是否符合业务预期。

6.7 用 MAT 定位泄漏的步骤

1. 打开 Heap Dump，先看 Leak Suspects，但不要机械接受自动结论。
2. 查看 Histogram，按 shallow heap 与 retained heap 排序。
3. 用 Dominator Tree 找支配大量对象的上层容器。
4. 对可疑对象执行 Path to GC Roots，通常排除 weak/soft references 后观察强引用链。
5. 结合业务语义判断：这个缓存、队列、ClassLoader 或线程为什么还持有对象。

- 对比两个时间点的 Dump，确认对象数量和保留大小是否持续增长。

概念区分：

- **Shallow Heap**：对象自身占用，不含它引用的对象。
- **Retained Heap**：如果该对象被回收，理论上可连带释放的对象总大小。
- **Dominator**：从 GC Root 到目标对象的所有路径都必须经过它；支配节点常是泄漏容器。

6.8 ThreadLocal 为什么容易泄漏

每个线程内部维护 ThreadLocalMap。Entry 的 key 对 ThreadLocal 是弱引用，但 value 是强引用。若业务代码丢失了 ThreadLocal key 的强引用，key 可能被 GC 变成 null，而 value 仍由长生命周期线程持有，直到 Map 在后续操作中清理或线程结束。

在线程池中，工作线程长期不结束，因此风险更高。正确模式：

```
private static final ThreadLocal<Context> CTX = new ThreadLocal<>();

public void handle() {
    try {
        CTX.set(buildContext());
        process();
    } finally {
        CTX.remove();
    }
}
```

还要注意上下文串请求：即使 value 很小，不清理也可能让下一个请求读取到上一个用户的信息，形成数据安全问题。

6.9 常见编码泄漏模式

- 静态 Map/Set 无淘汰。
- 本地缓存仅有过期时间，没有最大容量；或过期清理依赖访问，冷数据长期保留。
- 事件监听器注册后未注销。
- 定时任务持有外部对象引用。
- 自定义 ClassLoader、线程上下文 ClassLoader 阻止卸载。
- ThreadLocal 未 remove。
- 未关闭流、游标、响应体，导致 native 资源和连接泄漏。
- 队列生产速度长期大于消费速度。
- 把大对象捕获到长期存活的 Lambda、Future 或回调中。

6.10 一份高质量排查回答

我会先按 OOM 类型分流。Java heap space 看堆工作集和泄漏，保留 OOM 自动 Dump，并用 MAT 的 Histogram、Dominator Tree 和 Path to GC Roots 找出高 retained heap 的容器；Metaspace 看类加载数量和 ClassLoader 泄漏；Direct buffer memory 对比 RSS、堆和 NMT，检查直接缓冲区生命周期；unable to create new native thread 看线程数、栈大小、系统和容器 PID 限制。分析时要区分对象大、对象多和真正泄漏，并把 GC 日志、业务流量、队列长度及版本变更放在同一时间线上。

7. Spring 是如何解决循环依赖的？

7.1 先给边界结论

Spring 可以在一定条件下解决 **单例 Bean 的属性/Setter 注入循环依赖**，但不能把它概括成“Spring 能解决所有循环依赖”。常见边界：

- 构造器相互依赖通常无法自动解决，因为对象实例尚未创建就需要对方完整实例。
- prototype Bean 的循环依赖通常无法通过单例早期暴露机制解决。
- 是否允许循环引用还受容器配置和框架版本影响；在 Spring Boot 项目中不应依赖默认开关。
- 即使技术上可解决，循环依赖往往暴露职责耦合问题，应优先重构。

7.2 Bean 创建的简化流程

以单例 A 依赖 B、B 又依赖 A 为例：

1. 容器开始创建 A，调用构造器得到“原始 A 实例”。
2. 在属性填充前，注册一个可获取 A 早期引用的工厂。
3. 填充 A 的属性，发现需要 B，于是开始创建 B。
4. B 构造完成，填充属性时需要 A。
5. 容器发现 A 正在创建，从早期引用机制中取得 A 或 A 的早期代理，注入 B。
6. B 完成初始化并进入单例池。
7. A 获得 B，继续完成初始化，最终进入单例池。

核心是：先实例化，再属性注入；并在对象完全初始化之前有限度地暴露早期引用。

7.3 三级缓存分别是什么

面试中常说的三级缓存，位于单例注册管理逻辑中：

1. **一级缓存 singletonObjects**：存放已经完整创建并初始化完成的单例。
2. **二级缓存 earlySingletonObjects**：存放已经生成的早期单例引用，可能是原始对象，也可能是早期代理。
3. **三级缓存 singletonFactories**：存放 ObjectFactory，在需要时生成早期引用。

要避免错误表述：三级缓存不是普通业务缓存，也不是为了提升查询性能；它们用于管理单例创建状态和早期引用。

7.4 为什么需要第三级，二级是否足够

如果完全不考虑 AOP，只要在实例化后把原始对象直接放入二级缓存，从技术上可以解决部分属性注入循环依赖。

第三级的价值在于**延迟生成早期引用**：只有真的发生循环依赖、需要提前暴露时，才调用工厂获取引用。这个工厂可以让相关 BeanPostProcessor 决定是否返回代理对象，而不是无条件暴露原始对象。

因此更准确的回答是：

- 解决最简单的原始对象循环引用，二级结构可能足够。
- 为了兼容 AOP 代理、延迟暴露和保证早期引用的一致性，第三级工厂非常重要。

7.5 AOP 介入后为什么更复杂

假设 A 最终需要被事务或切面代理。如果 B 在循环依赖过程中拿到原始 A，而容器最终对外发布的是代理 A，就会出现同一个 Bean 的两个身份：

- B 内部持有原始对象，调用不经过切面。
- 其他 Bean 持有代理对象，调用会经过切面。

早期代理机制的目标是让循环依赖中的引用与最终暴露的引用保持一致。相关后处理器可以在早期引用阶段创建代理，并在后续初始化阶段避免重复代理。

这也是为什么“三级缓存纯粹为了性能”是错误答案。

7.6 构造器循环依赖为什么难解决

```
@Service
class A {
    A(B b) {}
}

@Service
class B {
    B(A a) {}
}
```

创建 A 之前必须先得到 B；创建 B 之前又必须先得到 A。此时连原始 A 都没有完成实例化，无法提前暴露引用。

可选方案：

- 重构职责，提取共同依赖 C。
- 改成事件发布/订阅，去除直接双向引用。
- 在非核心路径使用 @Lazy 或 ObjectProvider 延迟获取，但这只是打破初始化环，不一定解决设计耦合。

- 将需要代理调用的方法拆到独立服务。

构造器注入更容易在启动期暴露循环依赖，因此从设计质量看通常是优点，而不是缺点。

7.7 prototype 为什么通常解决不了

prototype Bean 每次请求都创建新实例，容器不会像单例那样在全局单例池中维护完整生命周期和早期引用。如果 A、B 都是 prototype 并相互依赖，创建链会不断递归，通常抛出正在创建相关异常。

单例依赖 prototype 还要注意：直接注入只会在单例创建时解析一次。若希望每次获得新 prototype，应使用 ObjectProvider、方法注入或代理作用域。

7.8 应不应该依赖循环依赖能力

不推荐。循环依赖常见设计问题：

- 两个服务职责边界不清。
- 领域操作被拆成相互回调。
- 为了复用一個私有方法而互相注入。
- 事务边界与业务边界错位。

重构方向：

- 提取第三个协调者或领域服务。
- 单向依赖：上层编排，下层提供能力。
- 使用领域事件解耦，但要明确同步/异步、一致性和失败补偿。
- 把事务方法拆到独立 Bean，避免自注入和循环代理。

7.9 一份面试标准表达

Spring 通过单例 Bean 的“实例化与初始化分离”以及早期引用机制解决部分属性注入循环依赖。一级缓存放完整单例，二级缓存放已经生成的早期引用，三级缓存放能够按需生成早期引用的 ObjectFactory。第三级的关键价值是兼容 AOP，让循环依赖注入的引用尽可能与最终代理一致。构造器循环依赖因为对象还没实例化通常无法解决，prototype 也不适用这套单例缓存机制。即使框架能处理，也应优先通过职责拆分消除循环依赖。

8. Spring 事务为什么会失效？

8.1 题目代码的结论

```
@Service
public class OrderService {

    public void createOrder() {
        saveOrder();
    }

    @Transactional
    public void saveOrder() {
        // 保存订单
    }
}
```

在默认的代理模式下，外部调用 `orderService.createOrder()` 时，`createOrder()` 本身没有事务；其内部通过 `this.saveOrder()` 直接调用目标对象方法，没有再次经过 Spring 代理，因此 `saveOrder()` 上的事务拦截通常不生效。

关键不是“同一个类不能加事务”，而是：事务通知应用在代理调用边界上，自调用绕过了代理。

8.2 Spring 声明式事务的调用链

外部调用通常是：

```
调用方 -> Spring 代理 -> TransactionInterceptor
        -> 开启/加入事务 -> 目标方法
        -> 提交或回滚 -> 返回
```

自调用则是：

```
目标对象的 createOrder() -> this.saveOrder()
```

中间没有代理拦截，自然没有新的事务语义。

8.3 private 方法为什么通常无效

代理需要拦截可代理的方法调用。`private` 方法不能被子类覆盖，且外部也无法通过代理接口调用，因此在常见代理模式下，把 `@Transactional` 放在 `private` 方法上不会形成预期事务边界。

同理还应注意：

- `final` 类或 `final` 方法对基于子类的代理有限制。
- JDK 动态代理主要基于接口方法。
- 具体限制取决于代理方式，但最佳实践是把事务放在对外的业务方法上，而不是依赖边缘代理技巧。

8.4 默认回滚规则

Spring 默认在以下情况回滚：

- 抛出 RuntimeException。
- 抛出 Error。

受检异常 Exception 默认不回滚，除非配置：

```
@Transactional(rollbackFor = Exception.class)
```

但不要机械地在所有方法上写 `rollbackFor = Exception.class`。应先设计异常分层：哪些表示业务可预期失败，哪些表示系统异常，哪些需要回滚，哪些允许部分提交。

8.5 catch 异常后为什么可能不回滚

```
@Transactional
public void saveOrder() {
    try {
        repository.insert();
        remoteCall();
    } catch (Exception e) {
        log.error("failed", e);
    }
}
```

异常被吃掉后正常返回，事务拦截器看不到异常，通常会提交。

解决方式：

- 记录后重新抛出合适异常。
- 明确调用 `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`，但会增加框架耦合。
- 使用 `TransactionTemplate`，在程序式事务中明确结果。
- 重新设计远程调用与本地事务的边界，避免长事务包住网络请求。

8.6 常见失效原因清单

1. 自调用绕过代理

解决：拆到另一个 Bean；或从代理对象调用。自注入虽然可能工作，但增加耦合，通常不是首选。

2. 方法不可代理

private/final、错误代理模式、类未被 Spring 管理等。

3. 对象不是容器 Bean

手动 `new OrderService()` 得到的对象不会被 Spring 代理。

4. 异常类型不触发默认回滚

受检异常未配置 `rollbackFor`。

5. 异常被捕获并吞掉

事务拦截器看到正常返回。

6. 数据库引擎或资源不支持事务

例如使用非事务型表、DDL 隐式提交、跨多个不受同一事务管理器控制的数据源。

7. 传播行为理解错误

调用方法已经在事务中，内部方法的 `REQUIRED` 只是加入现有事务，不会创建独立事务。

`REQUIRES_NEW` 需要通过代理调用，并要求事务管理器支持挂起外层事务。

8. 多线程/异步边界

事务上下文通常绑定当前线程。把任务提交到线程池后，新线程不会自动继承原事务。异步任务应建立自己的事务边界和幂等机制。

9. 回滚标记与异常被外层吞掉

内层参与同一事务的方法标记 `rollback-only`，外层捕获异常并继续，最终提交时可能得到 `UnexpectedRollbackException`。这不是事务“莫名失效”，而是事务已经被标记为只能回滚。

8.7 事务传播行为的理解

- `REQUIRED`：有事务则加入，没有则创建。最常用。
- `REQUIRES_NEW`：总是新建，外层事务被挂起。适合独立审计等少数场景，但会额外占用连接。
- `NESTED`：在支持保存点的资源事务中创建嵌套保存点，语义不同于独立事务。
- `SUPPORTS`：有事务则加入，没有则非事务执行。
- `MANDATORY`：必须在事务中，否则异常。
- `NOT_SUPPORTED`：以非事务方式执行，挂起现有事务。
- `NEVER`：存在事务则异常。

不要只背枚举，要说明连接占用。例如外层 `REQUIRED` 持有一个连接，内层大量 `REQUIRES_NEW` 需要额外连接，连接池不足时可能互相等待。

8.8 事务边界的生产设计

- 事务方法应围绕一个完整业务用例，而不是每个 DAO 方法都单独加事务。

- 临界区内只做必要数据库操作，避免远程调用、文件处理和长时间计算。
- 使用数据库唯一约束、行锁或乐观锁维护不变量。
- 事务提交后再发布外部副作用，可采用事务消息、Outbox 或事务同步回调。
- 对重试设计幂等，数据库死锁和瞬时故障可能导致事务重试。

8.9 修复题目代码

推荐拆分 Bean：

```
@Service
public class OrderApplicationService {
    private final OrderTxService orderTxService;

    public OrderApplicationService(OrderTxService orderTxService) {
        this.orderTxService = orderTxService;
    }

    public void createOrder() {
        orderTxService.saveOrder();
    }
}

@Service
public class OrderTxService {
    @Transactional(rollbackFor = Exception.class)
    public void saveOrder() {
        // 原子完成本地数据库状态变更
    }
}
```

这比通过 `AopContext.currentProxy()` 获取代理更清晰、更易测试。

8.10 一份高质量口述答案

Spring 声明式事务默认依赖代理，事务拦截发生在从代理进入目标方法的边界。同类内部调用是 this 调用，没有经过代理，所以方法上的 @Transactional 通常不生效。private/final 方法、手动 new 的对象、异常被吞、受检异常未配置回滚、多线程切换和传播行为误用也会导致预期与实际不一致。修复时优先把事务方法拆到独立 Bean，并把事务边界控制在必要的本地数据库操作内；跨进程副作用通过 Outbox、消息或补偿处理，而不是把远程调用包进长事务。

9. 数据库索引为什么会失效？

9.1 先纠正“索引失效”这个说法

面试中常说“索引失效”，实际可能有三种不同情况：

1. SQL 根本无法使用现有索引的有序结构完成定位。
2. 优化器认为使用索引成本更高，主动选择全表扫描。
3. 索引只用于部分条件或排序，剩余条件仍需回表或过滤。

因此不能仅凭 SQL 形式下结论，最终要结合数据分布、统计信息和 EXPLAIN ANALYZE。优化器选择全表扫描不一定是错误：当查询返回表中很大比例的数据时，顺序扫描可能更便宜。

9.2 联合索引与最左前缀

索引：

```
CREATE INDEX idx_user_status_time
ON user_order(user_id, status, create_time);
```

B+ 树中的键按 (user_id, status, create_time) 字典序排列。可直接利用的前缀包括：

- (user_id)
- (user_id, status)
- (user_id, status, create_time)

不能跳过最左列，直接把整个联合索引当作 (status, create_time) 的有序索引。现代 MySQL 可能使用 skip scan 等优化处理特定场景，但不能把它当作稳定、低成本的常规设计依据。

9.3 SQL 一：完整前缀 + 范围

```
SELECT * FROM user_order
WHERE user_id = 1001
AND status = 1
AND create_time > '2026-01-01';
```

通常可以有效使用联合索引：先按 user_id、status 精确定位，再对 create_time 做范围扫描。

但 SELECT * 往往需要回表：InnoDB 二级索引叶子节点保存索引列和主键值，找到候选主键后还要回聚簇索引读取其他列。若结果很多，随机回表成本可能使优化器选择其他计划。

9.4 SQL 二：缺少最左列

```
SELECT * FROM user_order
WHERE status = 1
AND create_time > '2026-01-01';
```

传统规则下无法利用该联合索引的最左前缀做高效定位，因为索引首先按 `user_id` 分组，同一 `status` 的记录分散在不同 `user_id` 段中。

可能出现的计划：

- 全表扫描。
- 全索引扫描后过滤。
- 优化器在满足条件时尝试 skip scan。

如果这是高频查询，应根据选择性和排序需求建立 (`status`, `create_time`) 等更匹配的索引，而不是指望原索引“顺便可用”。

9.5 SQL 三：对索引列做表达式

```
SELECT * FROM user_order
WHERE user_id + 1 = 1002;
```

通常不能直接使用普通 `user_id` 索引做等值定位，因为 B+ 树存储的是原始 `user_id` 的有序值，不是 `user_id + 1` 的结果。应改写为：

```
WHERE user_id = 1001
```

或者在确有需要时使用函数索引/生成列索引。类似问题还包括：

- `DATE(create_time) = '2026-01-01'`
- `CAST(user_id AS CHAR) = '1001'`
- 对列做计算或隐式类型转换

时间范围应写成：

```
WHERE create_time >= '2026-01-01 00:00:00'
AND create_time < '2026-01-02 00:00:00'
```

9.6 SQL 四：WHERE 与 ORDER BY 的关系

```
SELECT * FROM user_order
WHERE user_id = 1001
ORDER BY create_time;
```

索引顺序是 (`user_id`, `status`, `create_time`)。在 `user_id` 固定后，数据仍先按 `status` 分组，再在每个 `status` 内按 `create_time` 排序。因此整体结果不一定按 `create_time` 全局有序，通常无法直接利用该索引消除排序。

如果同时固定 `status`：

```
WHERE user_id = 1001 AND status = 1
ORDER BY create_time
```

则更可能直接按索引顺序输出。

如果业务高频需要“某用户所有状态按时间排序”，更合适的索引可能是 (user_id, create_time)。索引设计要服务具体查询组合，不存在一个联合索引覆盖所有排列。

9.7 范围查询之后的字段还能否使用

常见口诀是“范围列之后失效”，但需要更精确：

- 对于构造连续索引定位区间，范围条件之后的列通常不能进一步缩小扫描边界。
- 后续列仍可能通过 Index Condition Pushdown 在存储引擎层过滤，减少回表。
- 后续列也可能用于覆盖索引，避免回表。
- 某些 $\geq$ 、IN、多范围优化的具体计划取决于版本和数据分布。

例如索引 (a, b, c)：

```
WHERE a = 1 AND b > 10 AND c = 5
```

通常利用 a 和 b 确定扫描范围，c = 5 不能把范围进一步变成一个连续小区间，但可能在扫描二级索引时就过滤。

9.8 回表、覆盖索引和索引下推

回表

通过二级索引找到主键，再到聚簇索引读取完整行。结果行多、访问离散时成本很高。

覆盖索引

查询需要的列全部在某个索引中，存储引擎无需再访问聚簇索引。例如：

```
SELECT user_id, status, create_time
FROM user_order
WHERE user_id = ? AND status = ?;
```

如果所有列都包含在索引中，可能显示 Using index。

覆盖索引不是简单地把所有列都塞进索引。宽索引会增加空间、写放大、缓存压力和维护成本。

Index Condition Pushdown

在没有 ICP 时，存储引擎可能把索引命中的行逐个回表，再由 Server 层过滤。ICP 允许把部分只涉及索引列的条件下推到存储引擎，先过滤再回表，降低随机 I/O。

9.9 常见索引未被有效利用的原因

- 联合索引不满足最左前缀。
- 在索引列上使用函数、计算或隐式转换。

- LIKE '%keyword' 以前导通配符开始，普通 B+ 树无法定位前缀。
- OR 两侧索引条件差异大，成本估算后选择全表扫描；可评估 UNION ALL 或 index merge。
- 返回数据比例过高，优化器认为全表扫描更便宜。
- 统计信息过旧或数据倾斜严重。
- 排序方向、列顺序、范围条件与索引顺序不匹配。
- 字符集/排序规则或字段类型不一致导致转换。
- 小表扫描本来就比走索引更合理。

9.10 如何用 EXPLAIN 分析

至少关注：

- type: const、ref、range、index、ALL 等访问方式。
- possible_keys 与 key: 候选索引与实际索引。
- key_len: 实际使用的索引前缀长度，需结合字段类型理解。
- rows 与 filtered: 估算扫描行与过滤比例。
- Extra: Using index、Using index condition、Using filesort、Using temporary 等。

MySQL 8 可使用：

```
EXPLAIN ANALYZE
SELECT ...;
```

它会执行查询并展示实际行数和时间，比纯估算更可靠，但生产大查询要谨慎。

9.11 为什么索引不是越多越好

- 插入、更新、删除需要维护每个相关索引。
- 索引占磁盘和 Buffer Pool，挤压热点数据。
- 宽索引降低页内记录数量，增加树高度和 I/O。
- 过多相似索引增加优化器选择成本和运维复杂度。
- 唯一索引、二级索引还会影响锁范围和并发行为。

应通过慢 SQL、业务查询模式和数据分布设计索引，并定期识别重复、冗余和低使用率索引。

9.12 面试总结

联合索引按照列顺序形成字典序，最左前缀决定了能否快速确定连续扫描范围。对索引列做函数、跳过前导列或返回比例过高，都可能让普通索引无法高效定位或被优化器放弃。范围列之后的列不一定完全无用，它

还可能参与 ICP 或覆盖索引。判断不能只背口诀，要用 `EXPLAIN ANALYZE` 看实际扫描、回表、排序和行数估算，并结合写入成本设计最少但高价值的索引。

10. 如何保证接口幂等性？

10.1 幂等性的定义

幂等不是“接口只会被调用一次”，而是：**同一个业务请求被执行一次或多次，最终业务效果等价。**

支付回调、消息消费和网络重试天然可能重复。可靠系统不应依赖上游“绝不重复”，而应把重复视为正常输入。

先区分三种标识：

- 请求 ID：一次网络请求的追踪标识，重试时可能变化。
- 业务幂等键：代表同一业务动作，例如支付平台交易号、订单号 + 操作类型。
- 实体主键：数据库记录 ID，不一定等价于业务幂等键。

10.2 支付回调的推荐主流程

1. 验证签名、商户号、金额、币种和订单归属。
2. 提取稳定的业务幂等键，例如第三方支付流水号。
3. 在本地数据库事务中使用唯一约束防重复落库。
4. 锁定或条件更新订单状态，只允许合法状态迁移。
5. 记录原始回调和处理结果，便于审计。
6. 事务提交后返回成功；下游副作用通过 Outbox/MQ 异步触发。
7. 重复回调若业务已成功，应返回幂等成功，而不是报错诱发更多重试。

数据库设计示例：

```
CREATE TABLE payment_record (
  id BIGINT PRIMARY KEY,
  provider_txn_id VARCHAR(128) NOT NULL,
  order_id BIGINT NOT NULL,
  amount DECIMAL(18,2) NOT NULL,
  status VARCHAR(32) NOT NULL,
  UNIQUE KEY uk_provider_txn(provider_txn_id)
);
```

状态更新：

```
UPDATE orders
SET pay_status = 'PAID', paid_at = NOW(), version = version + 1
WHERE id = ?
AND pay_status = 'UNPAID';
```

受影响行数为 1 表示本次完成迁移；为 0 时应查询当前状态，区分“已支付的重复回调”和“非法状态”。

10.3 为什么唯一索引是最可靠的底线之一

应用层 select 后 insert 存在竞态：两个实例都查询不到，然后同时插入。数据库唯一约束在最终写入点维护不变量，能够跨线程、跨进程防重复。

正确处理方式是尝试插入并捕获唯一冲突，再读取已有记录进行语义判断。不要把唯一冲突一律当作系统错误。

唯一索引也不是万能的：

- 幂等键必须定义正确。
- 需要区分同一业务重试与不同业务请求。
- 后续外部副作用仍需幂等。
- 分库分表后，全局唯一约束可能无法由单库直接保证，需要路由同一分片或全局去重表。

10.4 Redis SET NX 的作用与局限

```
SET idempotent:<key> <value> NX EX 300
```

它适合：

- 在入口快速挡住短时间重复请求。
- 降低数据库压力。
- 表示“正在处理”，避免同一请求并发执行。

但不能把它当作唯一事实来源：

- Redis 成功后进程崩溃，业务可能没有落库，但 key 仍存在。
- key 过期后请求可能再次执行。
- Redis 数据可能因故障切换或淘汰丢失。
- 如果把结果状态只存在 Redis，难以审计和恢复。

推荐把 Redis 作为性能层，数据库唯一约束/状态机作为正确性底线。

10.5 分布式锁与幂等的区别

- 分布式锁解决“同一时刻只有一个执行者”。
- 幂等解决“重复执行不会造成重复效果”。

锁过期、进程暂停或网络分区都可能让第二个执行者进入，因此只加锁而没有幂等约束仍可能重复扣款。反过来，有可靠唯一约束时，很多场景无需分布式锁。

10.6 幂等 Token 方案

适合“提交订单”等由客户端主动发起的操作：

1. 客户端先向服务端申请一次性 token。
2. 服务端把 token 与用户、业务类型、有效期绑定。
3. 提交时携带 token。
4. 服务端原子地把 token 从 INIT 转为 PROCESSING，处理成功后保存结果。
5. 重复提交返回相同结果或当前处理状态。

应考虑：

- token 被抢用或跨用户使用。
- 处理超时后如何恢复。
- 客户端丢失首次响应时如何查询结果。
- token 状态与业务事务如何一致。

更完善的做法是建立幂等请求表：

idempotency_key	request_hash	status	response_snapshot	expire_at
-----------------	--------------	--------	-------------------	-----------

同 key 但请求参数摘要不同，应拒绝，避免用户误把同一 key 用于不同业务。

10.7 消息重复消费如何幂等

常见策略：

- 消息 ID/业务 ID 建唯一消费记录。
- 数据库条件更新，只允许状态向前迁移。
- 使用版本号或乐观锁。
- 消费记录与业务更新放在同一数据库事务。

错误做法：先在 Redis 标记已消费，再更新数据库。如果数据库失败，后续重试会被 Redis 阻挡，造成消息永久漏处理。

更可靠的本地事务示例：

```
BEGIN;
INSERT INTO consumer_dedup(message_id) VALUES (?)
-- 唯一冲突说明已处理
UPDATE business_table ...;
COMMIT;
ACK message;
```

如果事务提交后 ACK 丢失，消息会重投；唯一记录让重复消费安全返回。

10.8 Redis 成功但数据库失败怎么办

不要设计成“Redis 成功即代表业务成功”。可以采用状态机：

- PROCESSING：短 TTL，只代表有请求正在处理。
- 数据库事务提交成功后，将结果写入 Redis 或缓存响应。
- 数据库失败时删除/标记失败，允许重试。
- 如果进程崩溃，TTL 到期后可重新竞争；数据库唯一约束防止真正重复。

对于强审计场景，优先使用数据库幂等表，将 Redis 仅作为前置削峰。

10.9 状态机是高级答案的关键

订单不应只依赖“有没有处理过”布尔值，而应定义合法迁移：

```

CREATED -> PAYING -> PAID
          \-> FAILED
PAID -> REFUNDING -> REFUNDED
  
```

每个操作只接受允许的源状态。重复“支付成功”到达时，如果当前已经 PAID 且支付流水、金额一致，应幂等返回；如果当前为 REFUNDED 或流水不一致，则需要告警和人工核查。

幂等不是简单忽略重复，而是验证重复请求与历史结果是否一致。

10.10 一份面试答案模板

支付回调必须假设至少一次投递。我会用第三方交易号作为业务幂等键，先验签和校验金额，再在本地事务中通过唯一索引落支付记录，并用条件更新把订单从 UNPAID 迁移到 PAID。唯一冲突或影响行数为 0 时查询当前状态：若业务结果一致就返回成功，若参数冲突则告警。Redis SET NX 可以在入口减少并发重复，但不能替代数据库约束，因为 Redis 成功后数据库可能失败、锁也可能过期。事务后的通知通过 Outbox/MQ，消费者同样用业务 ID 去重，做到端到端幂等。

11. Redis 缓存与数据库如何保持一致？

11.1 先接受现实：缓存一致性通常是有窗口的

缓存与数据库是两个独立系统，在没有统一分布式事务的情况下，很难同时获得高性能、高可用和严格实时一致。工程目标通常是：

- 数据库作为最终事实来源。
- 缓存允许短暂不一致，但窗口有上界。
- 通过失效、重试、消息和兜底修复减少错误概率。
- 对余额、库存等强一致数据，不直接把缓存当最终判定依据。

11.2 Cache Aside 的标准读路径

1. 读缓存
2. 命中则返回
3. 未命中则读数据库
4. 写缓存并设置过期时间
5. 返回结果

伪代码：

```
Product getProduct(long id) {
    Product cached = cache.get(id);
    if (cached != null) return cached;

    Product db = repository.findById(id);
    cache.set(id, db, randomTtl());
    return db;
}
```

并发未命中时需防击穿，后文说明。

11.3 写路径：先更新数据库，再删除缓存

推荐常见模式：

1. 更新数据库并提交
2. 删除缓存

为什么通常不是“更新缓存”？

- 多个写线程可能乱序覆盖缓存。
- 缓存值可能由多表聚合，更新逻辑复杂。
- 删除让下一次读取按最新数据库重建，更简单。

仍存在失败窗口：数据库已提交但删除缓存失败，旧值会持续到 TTL。应配合：

- 缓存 TTL。
- 删除重试，且重试任务持久化。
- Binlog/CDC + MQ 驱动缓存失效。
- 监控删除失败和缓存版本。

11.4 为什么“先删缓存，再更新数据库”风险更大

时序：

1. 写线程删除缓存。
2. 读线程缓存未命中，读取旧数据库值。
3. 写线程更新数据库。
4. 读线程把旧值写回缓存。

结果是数据库新、缓存旧，并持续到 TTL。

延迟双删试图缓解：先删缓存，更新数据库，等待一段时间后再次删除。但“延迟多久”难以准确覆盖不同请求耗时，进程崩溃也可能漏掉第二次删除。它可以作为补充，不是严格一致性证明。

11.5 先更新数据库再删缓存也有并发窗口吗

有一种较少见但可能的时序：

1. 读线程缓存未命中，读到旧数据库值，但写缓存前被挂起。
2. 写线程更新数据库并删除缓存。
3. 读线程恢复，把旧值写回缓存。

发生条件较苛刻：读操作必须在写事务提交前读到旧值，且写缓存延迟跨过更新和删除。可以通过以下方式降低风险：

- 缓存较短 TTL。
- 写缓存时携带数据库版本号，只允许新版本覆盖旧版本。
- 使用逻辑过期 + 异步刷新。
- CDC 事件再次删除/刷新缓存。
- 对关键数据直接旁路缓存或读主库。

11.6 Binlog + MQ 的方案

流程：

1. 业务只更新数据库。
2. CDC 组件订阅 Binlog。
3. 把变更事件发送到消息队列。
4. 缓存服务按主键删除或刷新缓存。

优点：

- 缓存维护与业务代码解耦。
- 删除失败可以重试。
- 能统一处理多写入口。

难点：

- 事件延迟形成一致性窗口。
- 同一 key 的消息可能乱序，需要按 key 分区或比较版本。
- 消息可能重复，删除天然幂等，更新则要带版本。
- 全量重建、DDL、数据修复绕过正常链路时要有补偿。

11.7 缓存穿透

查询不存在的数据，每次都落到数据库。

解决：

- 参数校验和鉴权。
- 缓存空值，TTL 较短。
- Bloom Filter 过滤确定不存在的 key；要接受误判并处理数据新增同步。
- 防恶意枚举：限流、风控和接口设计。

空值缓存要区分“确实不存在”和“下游查询失败”，不能把临时故障缓存成不存在。

11.8 缓存击穿

某个热点 key 失效瞬间，大量请求同时查询数据库。

解决：

互斥重建

一个线程获得锁后查库和重建，其他线程等待、返回旧值或快速失败。锁必须有超时，且重建逻辑要做双重检查。

逻辑过期

缓存记录携带逻辑过期时间。过期后仍可暂时返回旧值，由一个线程异步刷新。适合允许短暂陈旧、强调可用性的场景。

提前刷新与热点永不过期

对可识别热点定时刷新，但“永不过期”仍需版本、主动失效和故障恢复，否则会永久脏读。

11.9 缓存雪崩

大量 key 同时失效或 Redis 整体不可用，引发数据库流量暴涨。

措施：

- TTL 加随机抖动。
- 分批预热和刷新。
- Redis 高可用与容量治理。
- 本地缓存作为短期兜底，但要限制容量与一致性风险。
- 网关限流、服务熔断、数据库保护。
- 降级返回静态数据或稍旧快照。

真正高可用答案不能只说“TTL 加随机数”，还要说明缓存整体故障时如何保护数据库。

11.10 热点 Key 与大 Key

热点 Key：访问集中在单 key，可能打满单分片或网络。可用：

- 本地多级缓存。
- 读副本或客户端副本，但要考虑一致性和故障切换。
- 将只读热点复制为多个逻辑 key，客户端随机读取。
- 合并请求、批量读取。

大 Key：单 key 值过大，导致网络、阻塞和删除成本。可用：

- 拆分结构。
- 使用异步删除。
- 限制单值大小和集合成员数量。
- 监控命令耗时与内存分布。

11.11 商品详情的推荐设计

- 数据库为真相源，写请求先提交数据库。
- 事务提交后投递 Outbox 事件，异步删除商品详情缓存。
- 缓存设置随机 TTL；热点商品使用逻辑过期和单飞重建。
- 缓存内容携带版本号，避免旧事件覆盖新值。
- 删除消息按商品 ID 分区以保持局部顺序，并允许重复消费。
- 缓存不可用时限流访问数据库，必要时返回短期本地快照。
- 价格、库存等强时效字段可拆分，不把一个巨大详情对象用同一一致性策略处理。

11.12 面试总结

我会采用 *Cache Aside*：读时缓存未命中查库回填；写时先提交数据库，再删除缓存。数据库成功、删缓存失败通过 TTL、持久化重试和 CDC/Binlog 消息兜底。并发读写仍存在极小窗口，可以用版本号、再次失效和逻辑过期缩短。热点 key 用单飞或逻辑过期防击穿，空值/Bloom Filter 防穿透，TTL 抖动和限流降级防雪崩。缓存只做性能层，关键业务正确性仍由数据库约束和状态机保证。

12. 消息队列如何保证不丢消息？

12.1 端到端分三段分析

消息可靠性必须分别回答：

1. 生产者到 Broker。
2. Broker 内部存储与复制。
3. Broker 到消费者以及业务处理。

只配置“消息持久化”并不能保证端到端不丢；只使用 ACK 也不能保证生产者已成功发送。

同时要接受：多数高可用消息系统更容易提供 **at-least-once**，也就是“不轻易丢但可能重复”。因此可靠性方案必须和幂等消费一起设计。

12.2 生产者阶段

发送确认

生产者应使用异步回调或确认机制，明确知道 Broker 是否接收并满足相应持久化条件。失败时：

- 有界重试。
- 指数退避和随机抖动。
- 记录失败消息到本地可靠存储或 Outbox。
- 对不可重试错误进入人工/补偿流程。

不要无限同步重试，否则 Broker 故障会阻塞业务线程并形成雪崩。

业务数据库与消息的一致性

典型问题：订单数据库提交成功，但消息发送失败。

推荐 Outbox：

```
BEGIN;
UPDATE orders ...;
INSERT INTO outbox(event_id, aggregate_id, payload, status) VALUES (...);
COMMIT;
```

后台发布者扫描/订阅 Outbox，把消息发送到 Broker，确认后标记已发送。即使发布者崩溃也能重试；重复发送由消息 ID 幂等处理。

其他方案包括事务消息或 CDC，但都要解释故障恢复路径，不能只说“使用分布式事务”。

12.3 Broker 阶段

要关注：

- 消息是否写入持久化日志，而不是仅在内存。
- 副本因子是否足够。
- 确认是否等待必要副本复制。
- 领导者故障切换时是否可能丢失未同步数据。
- 磁盘故障、容量不足和刷盘策略。

以 Kafka 为例，高可靠通常需要合理配置副本、acks=all、最小同步副本，并启用幂等生产者；但具体配置必须与集群规模和可用性目标匹配。副本不足时，是拒绝写入保证一致性，还是继续写入保证可用性，需要明确取舍。

以 RabbitMQ 为例，持久化消息配合耐久队列和 publisher confirms；对高可靠可使用 quorum queue，确认与复制语义要看队列类型和配置。

“写入磁盘”也不是绝对不丢。数据中心级灾难、同时多副本损坏、错误运维删除仍需跨地域备份、审计和恢复演练。

12.4 消费者阶段

推荐手动确认：

```
拉取消息 -> 执行业务事务 -> 提交成功 -> ACK
```

如果先 ACK 再处理，进程在处理前崩溃，消息会丢。

如果业务提交成功但 ACK 失败，Broker 会重新投递，造成重复。因此消费者必须幂等。

消费者本地事务示例：

```
BEGIN;
INSERT INTO consumed_message(message_id) VALUES (?);
UPDATE business_table ...;
COMMIT;
ACK;
```

唯一冲突表示已经处理，可直接 ACK。

12.5 自动确认还是手动确认

- 自动确认吞吐高、代码简单，但消息可能在业务完成前就被视为成功，可靠性较弱。
- 手动确认可以把确认放在业务成功之后，是关键业务的常见选择。
- 手动确认要处理超时、连接断开、重复投递和批量确认范围。

不应在收到消息后立即手动 ACK，这只是“手动执行自动确认”。

12.6 消息积压如何处理

先判断生产速度、消费速度和积压年龄，而不是立刻加消费者。

排查：

- 消费者是否报错、阻塞或频繁重平衡。
- 下游数据库/接口是否变慢。
- 单条消息处理是否出现毒丸消息。
- 分区数是否限制最大并行度。
- 是否存在热点分区或顺序约束。

处理策略：

1. 止住新增：限流、降级非关键生产。
2. 修复消费者故障。
3. 在分区/队列允许的范围内扩容消费者。
4. 优化批量处理、减少逐条远程调用。
5. 对历史积压启用临时专用消费者或离线回放。
6. 监控 lag、最老消息年龄和预计清空时间。

盲目扩容可能把数据库打垮，应把整条链路容量一起评估。

12.7 毒丸消息与死信

某条消息每次处理都失败，如果无限立即重试，会阻塞正常消息并形成 CPU/日志风暴。应：

- 区分可重试与不可重试错误。
- 延迟重试，采用退避。
- 超过阈值进入死信队列或停车场队列。
- 保留失败原因、原始消息、重试次数和版本。
- 提供人工修复与重放工具。

死信队列不是“永久垃圾桶”，必须有监控和处置流程。

12.8 如何保证顺序

消息队列通常只能在某个局部范围内保证顺序，例如 Kafka 单分区、RabbitMQ 单队列到单消费者的理想 FIFO。要保证同一订单事件顺序：

- 以订单 ID 作为分区键，使同一订单进入同一分区。
- 单分区内按顺序消费。
- 失败重试不能把后续消息无条件越过前序消息。
- 消费端仍应校验版本号/序列号，抵御重放和乱序。

全局顺序通常意味着单分区或单执行通道，吞吐会显著下降。应优先争取“同一业务实体有序”。

12.9 Exactly-once 应如何回答

“精确一次”通常有作用域和前提。Kafka 的幂等生产与事务可以在 Kafka 读写链路中提供更强语义，但当消费者还要更新外部数据库时，数据库提交与 Kafka offset/事务之间仍需要协调。

工程上更常见的策略是：

at-least-once 投递 + 幂等消费 + 可审计补偿

这比宣称“MQ 保证绝对不重复”更准确。

12.10 监控与演练

至少监控：

- 生产成功/失败/重试率。
- Broker 磁盘、水位、副本同步状态。
- 消费 lag、最老消息年龄。
- 消费成功、失败、重试、死信数量。
- 消息端到端延迟。
- 幂等冲突率。

定期演练：Broker 节点宕机、网络分区、消费者处理中崩溃、数据库成功但 ACK 失败、消息重复和乱序。没有故障演练的“可靠性配置”往往只存在于文档中。

12.11 面试总结

消息不丢要分生产、Broker、消费三段。生产端使用确认和有界重试，并通过 Outbox 或事务消息解决数据库提交后发送失败；Broker 端启用持久化、足够副本和必要的同步确认；消费端在业务事务提交后手动 ACK。提交成功但 ACK 失败会导致重投，所以消费必须通过消息 ID、业务唯一键和状态机幂等。顺序通常只保证到单分区或单业务 key，积压处理要同时评估消费者和下游容量，失败消息进入可监控、可重放的延迟重试和死信流程。

13. Redis 分布式锁应该如何实现？

13.1 题目代码的问题

```
redis.setnx("lock:order", "1");
redis.expire("lock:order", 30);

try {
    doBusiness();
} finally {
    redis.del("lock:order");
}
```

至少有五个问题：

1. SETNX 与 EXPIRE 不是原子操作。进程在两条命令之间崩溃，锁可能永久不过期。
2. 没有判断加锁是否成功，失败后也可能执行临界区。
3. 锁 value 固定为 1，无法识别锁的真正持有者。
4. 业务超过 30 秒后锁已过期，其他实例会获得锁；原持有者结束时又可能删除别人的锁。
5. Redis 主从复制和故障切换存在窗口，锁写入主节点但未复制时主节点宕机，新主节点可能不存在该锁。

13.2 单 Redis 实例的基本正确做法

加锁使用一条原子命令：

```
SET lock:order:<id> <unique-token> NX PX 30000
```

- NX：只有 key 不存在才设置。
- PX：同时设置毫秒级租约。
- value：每次加锁生成的唯一 token，例如 UUID + 进程/线程标识。

Java 伪代码：

```
String token = UUID.randomUUID().toString();
boolean locked = redis.set(key, token, SetArgs.Builder.nx().px(30_000));
if (!locked) {
    throw new BusinessException("resource is locked");
}
```

释放锁必须“比较 value 后删除”，且比较与删除要原子化：

```
if redis.call('get', KEYS[1]) == ARGV[1] then
    return redis.call('del', KEYS[1])
else
    return 0
end
```

这样旧持有者不会误删新持有者的锁。

13.3 为什么过期时间既必要又危险

没有过期时间，进程崩溃后锁永久存在。设置过期时间后，系统获得故障恢复能力，但锁只在租约内有效：

```
获得锁 -> 执行业务 -> 租约到期 -> 其他客户端获得锁
      \-> 旧客户端仍可能继续执行
```

因此 Redis 锁无法阻止一个发生长时间 GC、进程暂停或网络阻塞的旧客户端在租约到期后继续操作共享资源。

对严格正确性场景，需要 **fencing token（栅栏令牌）**：每次成功获得锁同时取得单调递增的序号，下游资源只接受比已见序号更大的写入。

```
客户端 A 获得 token=101，暂停
客户端 B 获得 token=102，写入资源，资源记录 102
客户端 A 恢复，携带 101 写入，被资源拒绝
```

前提是最终资源，例如数据库或存储服务，能够校验 token。只有 Redis 锁本身而下游不校验，无法完全解决过期持有者问题。

13.4 锁续期与看门狗

当业务时长不可精确预估时，可以定期续期：

- 只有持有相同 token 的客户端才能续期。
- 续期必须在租约到期前完成。
- 进程崩溃后停止续期，锁最终自动释放。

Redisson 的看门狗机制会在未显式指定固定租约、客户端仍存活时定期延长锁有效期。它简化了续期，但不是魔法：

- 长时间 Stop-The-World 超过续期间隔仍可能失锁。
- 网络隔离时续期失败。
- 业务代码必须处理锁失效后的重复执行风险。
- 看门狗不替代数据库唯一约束和幂等。

13.5 Redis 主从切换风险

典型时序：

1. 客户端 A 在主节点成功写入锁。
2. 锁尚未复制到从节点。
3. 主节点宕机，从节点提升为主。
4. 客户端 B 在新主节点成功获得同一把锁。

5. A、B 同时认为自己持有锁。

如果业务可以接受极低概率并发，并有数据库约束兜底，单主 Redis 锁可能足够。如果锁用于金融记账、唯一领导者或不可逆外部动作，则必须重新评估一致性模型，可能需要：

- 由数据库行锁/唯一约束完成互斥。
- 使用提供一致性共识的协调服务。
- 使用 fencing token 让资源端拒绝旧持有者。
- 重新设计为无锁幂等状态机。

13.6 Redlock 应如何看待

Redlock 尝试在多个相互独立的 Redis 主节点上获取多数锁，并根据耗时计算剩余有效期。它提高了对单实例故障的容忍，但其安全性依赖时钟、网络延迟、暂停和故障模型，业界存在长期讨论。

高级面试回答应避免两个极端：

- “Redlock 绝对安全，可以替代共识系统。”
- “Redlock 完全没用。”

更合理的表述是：根据业务失败代价和故障模型选择。对于正确性要求极高的锁，优先使用能在资源端校验 fencing token 的方案或一致性协调系统；对可重试、幂等、短临界区的业务，Redis 锁可以作为工程上高性价比的互斥手段。

13.7 哪些场景不应使用分布式锁

- 数据库唯一索引即可解决重复创建。
- 单条 SQL 条件更新即可完成状态竞争。
- 可以按业务 key 分区到单消费者串行处理。
- 操作天然可交换或可幂等，不需要互斥。
- 临界区包含不可控长时间外部调用。
- 需要跨地域、强一致领导者选举，却只使用单 Redis 锁。

优先级通常是：数据库约束/原子操作 > 状态机/幂等 > 分区串行化 > 分布式锁。

13.8 一个较完整的锁使用模板

```
String key = "lock:order:" + orderId;
String token = UUID.randomUUID().toString();
boolean locked = tryLock(key, token, Duration.ofSeconds(30));
if (!locked) {
    throw new BusinessException();
}
```

```
}  
  
try {  
    // 仍需数据库条件更新或唯一约束兜底  
    processOrderIdempotently(orderId);  
} finally {  
    unlockByLua(key, token);  
}
```

还应配套：

- 获取失败次数、等待时长、持锁时长指标。
- 锁续期失败告警。
- key 粒度与热点分析。
- 业务操作幂等。
- 超时与补偿流程。

13.9 面试总结

Redis 锁加锁要用 SET key token NX PX 保证设置值和过期时间原子化，token 必须唯一；释放要用 Lua 比较 token 后删除，防止旧持有者误删新锁。业务可能超过租约时可以续期或使用 Redisson 看门狗，但租约到期后旧客户端仍可能继续运行，所以严格正确性场景还要使用 fencing token 或数据库约束。Redis 主从异步复制在故障切换时可能产生双持有者，因此要根据失败代价决定是否应使用更强的一致性协调方案，很多业务其实用唯一索引、条件更新或幂等状态机更简单。

14. 设计一个高并发秒杀系统

14.1 目标与约束

题目条件：

- 库存 10,000。
- 瞬时请求 100 万。
- 不允许超卖。
- 每个用户只能购买一次。
- 系统需要降级与容灾能力。

首先要澄清成功率目标：100 万请求中只有 1 万可能成功，因此系统不需要把所有请求都送到数据库；正确设计应尽量在入口和缓存层快速淘汰 99% 以上的无效竞争。

系统目标可以拆为：

1. **安全性**：不超卖、不重复下单。
2. **高吞吐**：入口能承受瞬时洪峰。
3. **可用性**：局部故障时能够限流、降级、恢复。
4. **最终一致性**：Redis 预扣、消息和数据库库存最终对齐。
5. **可观测性**：能知道每个请求、库存和消息的状态。

14.2 总体流量路径

```

用户
-> CDN/静态页面
-> API 网关：鉴权、限流、风控、动态路径
-> 秒杀服务
    -> Redis Lua：资格校验 + 一人一次 + 预扣库存
    -> MQ：成功资格事件
-> 订单消费者
    -> 数据库本地事务：创建订单 + 扣减/确认库存
    -> 结果缓存
-> 客户端轮询/推送查询结果
  
```

原则是把成本低、可快速拒绝的判断放在前面，把数据库写入放到削峰后的异步链路。

14.3 CDN 与静态化

- 活动页面、商品图片、规则说明放 CDN。
- 秒杀开始时间、商品基础信息可静态化，但最终资格以服务端时间与状态为准。

- 页面按钮灰显只是体验优化，不能作为安全边界。
- 避免页面加载就请求大量动态接口。

静态化减少源站流量，但攻击者可直接调用接口，因此网关和服务端校验仍必须完整。

14.4 网关限流与分层削峰

限流维度：

- 全局活动限流。
- 单商品限流。
- 单用户/设备/IP 限流。
- 单机并发与线程池队列限流。

算法可使用令牌桶或漏桶。入口容量应根据 Redis、MQ 和下游可持续吞吐设定，不是根据用户总量设定。

示例：如果秒杀服务与 Redis 集群压测可稳定处理 20 万请求/秒，网关不应放任 100 万请求同时进入后再“看系统能不能扛住”。其余请求可快速返回“活动火爆，请稍后查询结果”或直接失败。

14.5 验证码、动态路径与风控

作用不是绝对防作弊，而是提高自动化脚本成本并削平请求时间：

- 活动前完成验证码或行为验证。
- 通过后为用户签发短期、绑定用户和商品的动态路径/令牌。
- 服务端验证签名、过期时间和重放。
- 结合设备指纹、异常频率和黑名单。

不能把动态 URL 当作秘密长期暴露，也不能只在前端校验。

14.6 Redis Lua 原子预扣

需要在一个原子脚本中完成：

1. 检查活动状态。
2. 检查用户是否已获得资格。
3. 检查库存是否大于 0。
4. 扣减库存。
5. 记录用户资格或请求 ID。

伪 Lua：

```
-- KEYS[1] stock key
-- KEYS[2] purchased user set
-- ARGV[1] user id

local stock = tonumber(redis.call('get', KEYS[1]) or '-1')
if stock <= 0 then
    return 1 -- sold out
end

if redis.call('sismember', KEYS[2], ARGV[1]) == 1 then
    return 2 -- duplicate
end

redis.call('decr', KEYS[1])
redis.call('sadd', KEYS[2], ARGV[1])
return 0
```

实际设计还要考虑用户集合过大、活动过期清理、集群槽位。Redis Cluster 下脚本涉及的 key 必须在同一 hash slot, 可使用相同 hash tag, 例如:

```
seckill:{activityId}:stock
seckill:{activityId}:users
```

14.7 为什么 Redis 预扣仍不能单独保证不超卖

Redis 可能重启、故障切换、数据恢复或人工修复; MQ 和数据库也可能失败。最终数据库仍需约束:

```
UPDATE sku_stock
SET available = available - 1
WHERE sku_id = ?
AND available > 0;
```

或者数据库保存活动总库存和已售计数, 通过条件更新保证不为负。创建订单还应有:

```
UNIQUE KEY uk_activity_user(activity_id, user_id)
```

Redis 负责高性能资格预判, 数据库约束负责最终事实的一致性底线。

14.8 MQ 削峰与订单异步化

Lua 成功后发送“获得下单资格”消息。客户端不要同步等待数据库订单创建, 而是立即得到请求受理 ID, 随后查询:

```
PROCESSING / SUCCESS / FAILED / SOLD_OUT
```

生产可靠性:

- Redis 成功但消息发送失败, 不能让资格永久丢失。
- 可以把资格事件写入 Redis Stream, 再由可靠转发器投递 MQ; 或使用本地/分布式事件日志。
- 定期扫描 Redis 中已预扣但未创建订单的记录进行补发。

消费可靠性:

- 订单消息可能重复, 数据库唯一索引防重复订单。
- 消费事务成功后 ACK。

- 失败进入延迟重试，最终进入人工/补偿队列。

14.9 Redis 成功、数据库失败时如何补偿

失败类型不同，策略不同：

可重试失败

数据库短暂超时、死锁等：消息重试，不立即归还 Redis 库存，避免同一库存被多次分配。

不可重试失败

用户资格不合法、商品配置错误等：标记失败，并通过原子脚本归还 Redis 库存、移除用户资格或记录失败状态。

长时间未知

进入对账任务：比较 Redis 资格、MQ 事件、订单表和库存流水，按业务 ID 修复。

补偿操作必须幂等，不能重复归还库存。可为每次预扣生成 reservation ID，并记录状态：

```
RESERVED -> CONFIRMED
          -> RELEASED
```

只有 RESERVED 能转为 RELEASED，避免重复释放。

14.10 一人一单如何保证

至少两层：

1. Redis Lua 中检查用户集合，快速拒绝重复请求。
2. 数据库唯一索引 (activity_id, user_id)，防止 Redis 故障或重复消息造成重复订单。

不要只使用分布式锁。100 万请求下为每个用户获取锁增加开销，且锁仍存在过期与故障问题；唯一约束更直接。

14.11 热点 Key 的处理

库存 key 是极端热点。方案包括：

单 key + 高性能 Redis

对于 1 万库存、活动时间短的场景，单 Redis 主节点的单 key 原子操作可能已足够，先以压测数据为准，不要过早复杂化。

库存分桶

把 10,000 库存拆成多个 bucket，请求按哈希选择桶。某桶售罄时可尝试少量其他桶。难点是：

- 总库存聚合。
- 桶间不均衡。
- 多桶重试放大。
- 一人一次需要全局或一致路由。

请求前置分片

按用户或商品把流量分发到多个逻辑处理分片，每个分片持有部分配额。适合超大规模平台，但补货、回收和分片故障恢复更复杂。

高级答案应强调：是否拆热点要由单节点吞吐、网络和活动规模压测决定。

14.12 数据库表与库存模型

推荐保留库存流水，而不是只看一个数字：

```
activity_stock(activity_id, total, reserved, sold, version)
seckill_order(activity_id, user_id, order_id, reservation_id, status)
stock_reservation(reservation_id, activity_id, user_id, status)
```

不变量：

```
total = available + reserved + sold + released/expired 的可解释关系
```

实际字段可以更简洁，但必须能够审计“库存为什么变成现在这个数”。

14.13 降级与容灾

Redis 故障

- 立即关闭秒杀入口或进入只读“活动火爆”降级，不能把 100 万请求直接打到数据库。
- 切换 Redis 后根据持久化与事件日志恢复库存状态。

MQ 故障

- 暂停发放新资格，或把事件写入可靠本地/Redis Stream 缓冲。
- 不能在内存队列无限堆积。

数据库故障

- 停止新资格或限制发放速度。
- 已获得资格进入 PROCESSING，恢复后重放。
- 不要立即把所有预扣库存归还，避免数据库短暂故障造成超卖式重复分配。

服务实例故障

- 请求幂等，实例无状态。
- 资格与处理状态存外部可靠存储。
- 通过负载均衡摘除并自动扩缩容。

14.14 可观测性

核心指标：

- 网关请求量、限流量、风控拒绝量。
- Lua 成功、售罄、重复、错误数量与延迟。
- Redis CPU、网络、慢命令、热点 key。
- MQ 发送失败、积压、最老消息年龄。
- 订单创建成功/失败/重复率。
- Redis 预扣数、数据库订单数、补偿数之间的差异。
- 用户端 PROCESSING 停留时间。

每个资格请求必须有 request ID/reservation ID，贯穿日志、消息和订单表。

14.15 容量估算示例

假设入口瞬时 100 万请求集中在 5 秒内，即 20 万 QPS。若网关只放行 5 万 QPS 到秒杀服务，Redis Lua 平均 1 ms 并不代表只需 50 并发，因为还要考虑连接、网络和尾延迟。应做分层压测：

1. 单 Redis 节点 Lua 吞吐与 P99。
2. 秒杀服务实例的连接池和序列化开销。
3. 网关与风控链路。
4. MQ 的持续写入能力。
5. 数据库消费者的稳定订单创建 TPS。

秒杀成功消息最多约 1 万条，数据库只需要处理成功资格和少量补偿，而不是 100 万原始请求。这正是架构削峰的核心价值。

14.16 面试总结

我会把 100 万请求的大部分挡在 CDN、网关限流和风控层。进入秒杀服务后，用 Redis Lua 原子完成活动校验、一人一次和预扣库存，成功者写入可靠事件并立即返回受理 ID，订单通过 MQ 异步创建。数据库使用 `available > 0` 条件更新和 `(activity_id, user_id)` 唯一索引作为最终不超卖、不重复的底线。Redis 预扣与

数据库失败通过 *reservation* 状态机、延迟重试和对账补偿闭环。*Redis*、*MQ* 或数据库故障时宁可关闭入口或返回处理中，也不能让洪峰绕过缓存直击数据库。

15. 编程题：实现线程安全的 LRU 缓存

15.1 设计思路

要让 get 和 put 平均 $O(1)$ ，需要：

- `HashMap<K, Node<K,V>>`： $O(1)$ 定位节点。
- 双向链表： $O(1)$ 把访问节点移动到头部，以及淘汰尾部节点。
- 一把互斥锁：保护 Map 与链表的复合不变量。

链表约定：

`head <-> 最近使用 ... 最久未使用 <-> tail`

使用两个哨兵节点可以减少头尾判空分支。

15.2 完整实现

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.locks.ReentrantLock;

public final class LruCache<K, V> {

    private final int capacity;
    private final Map<K, Node<K, V>> index;
    private final Node<K, V> head;
    private final Node<K, V> tail;
    private final ReentrantLock lock = new ReentrantLock();

    public LruCache(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("capacity must be positive");
        }
        this.capacity = capacity;
        this.index = new HashMap<>(Math.max(16, capacity * 2));
        this.head = new Node<>(null, null);
        this.tail = new Node<>(null, null);
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        Objects.requireNonNull(key, "key");
        lock.lock();
        try {
            Node<K, V> node = index.get(key);
            if (node == null) {
                return null;
            }
            moveToFront(node);
            return node.value;
        } finally {
            lock.unlock();
        }
    }

    private void moveToFront(Node<K, V> node) {
        index.remove(node.key);
        node.prev.next = node.next;
        node.next.prev = node;
        node.prev = head;
        head.next = node;
        index.put(node.key, node);
    }
}
```

```

        lock.unlock();
    }
}

public void put(K key, V value) {
    Objects.requireNonNull(key, "key");
    lock.lock();
    try {
        Node<K, V> existing = index.get(key);
        if (existing != null) {
            existing.value = value;
            moveToFront(existing);
            return;
        }

        Node<K, V> created = new Node<>(key, value);
        index.put(key, created);
        addAfterHead(created);

        if (index.size() > capacity) {
            Node<K, V> evicted = removeLast();
            index.remove(evicted.key);
        }
    } finally {
        lock.unlock();
    }
}

public int size() {
    lock.lock();
    try {
        return index.size();
    } finally {
        lock.unlock();
    }
}

public boolean containsKey(K key) {
    Objects.requireNonNull(key, "key");
    lock.lock();
    try {
        return index.containsKey(key);
    } finally {
        lock.unlock();
    }
}

private void moveToFront(Node<K, V> node) {
    unlink(node);
    addAfterHead(node);
}

private void addAfterHead(Node<K, V> node) {
    Node<K, V> first = head.next;
    node.prev = head;
    node.next = first;
    head.next = node;
    first.prev = node;
}

private void unlink(Node<K, V> node) {
    Node<K, V> before = node.prev;

```



```

        Node<K, V> after = node.next;
        before.next = after;
        after.prev = before;
        node.prev = null;
        node.next = null;
    }

    private Node<K, V> removeLast() {
        Node<K, V> last = tail.prev;
        if (last == head) {
            throw new IllegalStateException("cache is empty");
        }
        unlink(last);
        return last;
    }

    private static final class Node<K, V> {
        private final K key;
        private V value;
        private Node<K, V> prev;
        private Node<K, V> next;

        private Node(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }
}

```

15.3 为什么 get 也必须加写锁

LRU 的 get 不只是读取 value，还要把节点移动到链表头部。因此它会修改链表的 prev/next，是写操作。如果使用读写锁并让多个 get 同时持读锁，多个线程会并发修改链表，破坏结构。若每次 get 都获取写锁，那么读写锁退化为互斥锁，还增加复杂度。

因此对严格访问顺序 LRU，一把 ReentrantLock 或 synchronized 通常更合适。

15.4 能否优化并发度

可以，但会牺牲严格 LRU 或增加复杂度：

- 分段 LRU：按 key 哈希到多个独立缓存，每段各自淘汰，整体不是严格全局 LRU。
- 访问记录异步批处理：读路径只记录访问，后台更新淘汰结构，顺序是近似的。
- 使用成熟缓存库，如 Caffeine，采用更先进的命中率与并发算法。

高级面试中应说明：生产环境通常不建议手写缓存淘汰库；这道题主要考察数据结构和并发不变量。

15.5 API 设计中的细节

当前 get 返回 null 表示未命中，因此无法区分“key 不存在”和“缓存的 value 本来就是 null”。可以：

- 禁止 null value。

- 返回 `Optional<V>`。
- 提供 `containsKey`，但两个调用之间仍可能变化。

还要考虑：

- 淘汰回调是否在锁内执行。通常不应在锁内调用外部代码。
- `value` 是否可变。缓存线程安全不代表 `value` 自身线程安全。
- 过期策略、最大权重、加载异常和统计指标。

15.6 关键不变量

任何时刻都应满足：

1. `Map` 中的每个节点恰好在链表中出现一次。
2. 链表中的每个业务节点都能通过 `Map` 找到。
3. `head.prev == null`、`tail.next == null`。
4. `head.next.prev == head`，`tail.prev.next == tail`。
5. 缓存大小不超过 `capacity`。

锁的意义是让一次 `Map` + 链表的复合更新对其他线程不可见为“半完成状态”。

16. 接口变慢但 CPU 和内存正常，如何排查？

16.1 结论先行

CPU、内存、GC 正常只说明 JVM 没有明显算力或堆压力，不代表系统健康。接口从 100 ms 变成 3 秒，最常见原因是等待：数据库、连接池、下游 HTTP、DNS、网络、锁、线程池队列、磁盘或限流。

排查应从端到端延迟分解开始，而不是在单机上盲查。

16.2 先确认问题范围

需要快速回答：

- 所有接口慢，还是单个接口慢？
- 所有实例慢，还是单实例慢？
- 所有用户/地域慢，还是特定网络慢？
- P50 变慢还是只有 P99？
- 从什么时间点开始，是否与发布、流量、数据量或依赖变更重合？
- 服务端耗时变慢，还是客户端到网关的网络耗时变慢？

把指标按版本、实例、可用区、请求类型和下游依赖切分，能快速缩小范围。

16.3 使用链路追踪分解时间

一条请求可拆为：

```
网关排队
+ 应用线程池排队
+ 业务代码 CPU
+ 数据库连接等待
+ SQL 执行
+ 下游连接等待
+ DNS/建连/TLS
+ 下游处理
+ 日志/磁盘
+ 响应传输
```

分布式追踪中的 Span 应包含：

- 客户端/服务端耗时。
- 数据库语句或模板标识。
- 连接池等待。
- 下游状态码和重试次数。

- 超时发生阶段。

如果没有追踪，先用访问日志记录总耗时，再在关键步骤增加结构化耗时，但避免高峰期打印巨量同步日志。

16.4 数据库方向

检查：

- 慢 SQL 日志与执行计划是否变化。
- 数据量增长导致索引选择变化。
- 锁等待、死锁、长事务。
- Buffer Pool 命中率、磁盘 I/O。
- 数据库 CPU、连接数、活跃会话。
- 主从延迟或跨地域访问。

连接池指标尤其重要：

- active、idle、max。
- 获取连接等待时间。
- 获取超时次数。
- 连接泄漏检测。

应用 CPU 很低可能是因为大量请求线程都在等待数据库连接。

16.5 下游 HTTP/RPC

关注：

- DNS 解析时间。
- 建立 TCP 连接、TLS 握手时间。
- 连接池是否复用，是否耗尽。
- 下游 TTFB 和完整响应时间。
- 重试次数、退避策略。
- 超时是否分为 connect/read/write/overall。

典型故障：下游 1 秒超时，客户端自动重试 2 次，总耗时变成约 3 秒，而本服务 CPU 仍很低。要在追踪中记录每次 attempt，而不是只看到一次 3 秒调用。

16.6 DNS 与网络

DNS 问题可能表现为间歇性秒级延迟。排查：

```
dig <host>
nslookup <host>
getent hosts <host>
curl -w 'dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n' -o /dev/null -s https://example.com
```

网络方向看：

- 丢包、重传、RTT。
- NAT/连接跟踪表耗尽。
- 跨可用区或跨地域路由变化。
- MTU、代理、服务网格 sidecar。
- 端口耗尽和大量 TIME_WAIT。

工具可用 ss -s、sar -n DEV 1、mtr、tcpdump，但抓包需注意数据安全与性能。

16.7 线程池与队列积压

即使 CPU 低，任务可能在队列中等待：

- Web 容器线程池已满。
- 业务线程池队列变长。
- 异步 Future 最终又同步 get。
- 下游慢导致工作线程长期占用。

必须分别记录：

```
submit_time -> start_time: 排队时间
start_time -> end_time: 执行时间
```

如果只记录执行时间，可能误以为业务只耗时 100 ms，但请求在队列中等了 2.9 秒。

16.8 锁等待

CPU 低且延迟高，也可能是大量线程等待锁：

- synchronized 监视器竞争。
- ReentrantLock/AQS park。
- 数据库行锁。
- 分布式锁等待。

连续线程栈与 JFR 可以定位。重点找持锁线程在做什么：若它在锁内访问远程服务，任何下游抖动都会放大成全局串行阻塞。

16.9 Redis 慢查询

检查：

- 单命令延迟和慢日志。
- 大 key、热 key。
- 阻塞命令或 Lua 脚本执行过久。
- 网络带宽与连接池。
- 集群重定向、故障切换。
- Redis 实例 CPU、内存碎片和持久化影响。

不要只看 Redis 平均延迟。一个大 key 的偶发操作可能主要影响 P99。

16.10 日志与磁盘 I/O

- 同步日志写入慢盘。
- 日志队列已满后回退为调用线程写入。
- 磁盘空间接近满或 inode 耗尽。
- 容器日志驱动阻塞。
- 打印超大 JSON、异常堆栈或二进制内容。

检查 `iostat -x 1`、磁盘利用率、await、日志框架队列和丢弃策略。

16.11 Safepoint 与非 GC 停顿

GC 正常不等于没有 JVM 停顿。还可能有：

- 到达 safepoint 时间过长。
- 类重定义、偏向撤销等旧版本行为。
- 大量线程导致 safepoint 协调成本。
- JIT 编译或 deoptimization 抖动。

统一日志中可以启用 safepoint 信息，JFR 也能查看停顿事件。

16.12 系统化排查顺序

1. 看入口 SLA: QPS、错误率、P50/P95/P99。
2. 按实例、版本、地域和接口切分。
3. 看链路追踪最长 Span。
4. 检查线程池排队与连接池等待。
5. 检查数据库慢 SQL、锁和连接。
6. 检查下游延迟、重试、DNS、网络。
7. 检查锁、Redis、日志、磁盘与 safepoint。
8. 对比变更时间线并做回滚/隔离验证。

16.13 一份面试总结

CPU 和内存正常时，我会优先怀疑等待而不是计算。先确认慢的是单接口、单实例还是全链路，再用 tracing 把 3 秒拆成线程池排队、连接池等待、SQL、下游 HTTP、DNS/建连、锁和日志等阶段。重点看 P99、重试次数、连接获取时间和任务排队时间，因为这些不会直接反映为高 CPU。若没有 tracing，就通过访问日志和关键步骤计时快速建立分段耗时。定位后要同步做超时、限流、隔离、回滚或降级，避免慢依赖占满线程和连接形成级联故障。

17. 综合项目题：如何回答“做过最复杂的项目”

17.1 面试官真正想判断什么

这道题不是让候选人朗读系统模块，而是判断：

- 是否真正负责过核心问题。
- 能否量化规模和目标。
- 是否理解方案约束与取舍。
- 是否经历过线上故障并形成闭环。
- 是否能区分“团队做的”和“自己做的”。
- 能否从业务结果反推技术价值。

17.2 推荐回答结构

第一段：30 秒背景

这是一个面向 XXX 用户的 XXX 系统，核心链路是 A -> B -> C。
日常 QPS 大约 X，峰值 Y，数据量 Z，SLA 为 P99 小于 N ms、可用性 N 个 9。
我负责的是其中的 M 模块和跨团队的 N 项改造。

不要说“并发很高、数据很多”，必须给数量级和口径：入口 QPS、单实例 QPS、日增数据、总数据、峰值持续时间。

第二段：核心挑战

只选 1~2 个最能体现能力的问题：

- 热点库存与超卖。
- 账务一致性。
- 亿级数据查询。
- MQ 积压与重复消费。
- 大促容量治理。
- 多租户隔离。

说明为什么难，而不是堆技术名词。

第三段：方案与取舍

按照：

约束 -> 候选方案 -> 最终选择 -> 关键机制 -> 为什么没选其他方案

例如：

我们没有直接使用全局分布式锁，因为热点订单会串行化且锁租约难以覆盖远程调用。最终用数据库唯一约束保证一人一单，Redis Lua 做前置资格预扣，MQ 异步削峰，数据库条件更新作为库存底线。

第四段：结果

量化：

- P99 从 1.2 s 降到 180 ms。
- 单实例吞吐从 800 提升到 2,500 QPS。
- 大促数据库峰值降低 85%。
- 消息积压恢复时间从 2 小时降到 15 分钟。
- 故障恢复 RTO/RPO 改善。

没有数据时可以诚实说明观测口径，不要编造。

第五段：反思

说清楚现在重做会改变什么：

- 更早建设压测和容量模型。
- 用 Outbox 替代业务代码双写。
- 拆分强一致字段与弱一致详情缓存。
- 增加自动对账与回放平台。

反思不是否定原方案，而是体现认知升级。

17.3 示例回答：高并发订单与库存系统

我参与的是电商订单履约平台，核心链路包括下单、库存预占、支付确认和履约事件。日常入口约 8,000 QPS，大促峰值约 45,000 QPS，订单表规模数十亿级，核心下单链路要求 P99 小于 300 ms。我负责库存预占服务、订单事件可靠性和大促容量治理。

当时最大问题是热门商品形成热点，旧方案在数据库用悲观行锁串行扣库存，流量升高后锁等待超过 2 秒，并导致应用线程池和连接池一起耗尽。我们首先明确不超卖和一人一单由数据库最终约束保证，但把大部分竞争前移：入口按活动和用户限流，Redis Lua 原子完成资格与预扣，成功事件进入 MQ，订单消费者用唯一键和条件更新落库。为解决 Redis 成功但消息失败，我们引入 reservation ID 和事件补偿扫描；数据库提交后 ACK，重复消息通过唯一消费记录幂等。没有采用单一全局分布式锁，因为它会把热点串行化，并且租约过期仍不能成为最终正确性保证。

上线后数据库峰值写入下降约 80%，下单 P99 稳定在 220 ms 左右，重复订单由数据库唯一约束降为 0；故障演练中，即使 MQ 暂停 10 分钟，资格事件也能恢复重放。一次严重事故是某次配置错误让消费者重试无

退避，失败消息形成重试风暴。我们先暂停该消费组、扩容隔离数据库连接池并限流生产，然后根据 message ID 修复并回放。之后增加了指数退避、死信停车场、最老消息年龄告警和一键暂停功能。

如果重新设计，我会更早把库存 reservation 建成显式状态机，并用统一 Outbox/CDC 平台替代多个业务模块自建补偿任务；同时把容量模型和故障演练纳入发布门禁，而不是只在大促前集中压测。

17.4 常见低分回答

- 只画架构图，不说本人负责内容。
- 堆叠 Spring Cloud、Redis、Kafka、ES 等名词，不解释为什么使用。
- 只说成功方案，不说故障和边界。
- “我们用了分布式锁保证一致性”但说不清超时、续期和数据库兜底。
- 数据口径混乱，把日请求量当 QPS。
- 把同事或平台能力全部描述成个人成果。

17.5 追问准备表

追问	应准备的内容
峰值怎么测出来	监控口径、压测模型、持续时长、单实例与集群
为什么选这个中间件	数据规模、延迟、可靠性、团队运维能力、替代方案
一致性怎么保证	不变量、事务边界、幂等键、补偿和对账
最大事故是什么	时间线、影响面、止损、根因、长期改进
方案缺点是什么	成本、复杂度、一致性窗口、运维风险
你具体做了什么	设计、编码、压测、上线、指标、协作边界

18. 附录：命令、公式与检查清单

18.1 JVM 诊断命令速查

```
# 列出 Java 进程
jcmd -l

# 线程栈
jcmd <pid> Thread.print -l

# 类直方图
jcmd <pid> GC.class_histogram

# Heap Dump
jcmd <pid> GC.heap_dump filename=/data/heap.hprof

# JVM 参数与系统属性
jcmd <pid> VM.flags
jcmd <pid> VM.system_properties

# 原生内存（需启动时开启 NM1）
jcmd <pid> VM.native_memory summary

# 启动 60 秒 JFR
jcmd <pid> JFR.start name=diag settings=profile duration=60s filename=/data/diag.jfr

# 高 CPU 线程
top -Hp <pid>
printf '%x\n' <tid>
```

18.2 Linux 性能命令速查

```
uptime
vmstat 1
mpstat -P ALL 1
pidstat -u -t -p <pid> 1
iostat -x 1
sar -n DEV 1
ss -s
free -m
df -h
df -i
```

18.3 容量估算公式

Little 定律

平均在途数量 $L = \text{吞吐率 } \lambda \times \text{平均停留时间 } W$

适用于稳定系统和一致统计口径。不能用平均值替代尾延迟容量设计。

线程数经验公式

线程数 $\approx \text{CPU 核数} \times \text{目标利用率} \times (1 + \text{等待时间} / \text{计算时间})$

只适合作为初始值，最终要通过压测校准，并受下游连接池和限流约束。

队列容量

队列容量 $\approx$ 可接受排队时长 $\times$ 可持续消费速率

必须配合拒绝和降级；队列不是免费容量。

可用性错误预算

月度允许不可用时间 $\approx$ 月总时间 $\times$ (1 - SLA)

例如 99.9% 每月约允许 43 分钟不可用；应以公司实际统计口径为准。

18.4 高级 Java 面试回答检查清单

回答每道题时，尽量覆盖：

- 是否先给明确结论。
- 是否说明版本和适用前提。
- 是否解释底层机制而非只背名词。
- 是否说明失败路径和并发边界。
- 是否提供生产选型与监控指标。
- 是否说明方案缺点和替代方案。
- 是否用数据、代码或命令支撑。

18.5 参考资料

以下资料用于校验公开语义和现代生产建议。源码实现细节应以实际使用版本为准。

1. Oracle Java SE API: java.util.concurrent、ConcurrentHashMap、ThreadPoolExecutor、原子类与锁。 <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/package-summary.html>
2. Oracle Java SE 21 Troubleshooting Guide: jcmd、JFR、NMT、Heap Dump 与线程诊断。 <https://docs.oracle.com/en/java/javase/21/troubleshoot/diagnostic-tools.html>
3. Spring Framework Reference: 依赖注入与循环依赖。 <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>
4. Spring Framework Reference: AOP 代理机制与 self invocation。 <https://docs.spring.io/spring-framework/reference/core/aop/proxying.html>

5. Spring Framework Reference: @Transactional 语义与默认回滚规则。
<https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html>
6. MySQL Reference Manual: Multiple-Column Indexes。
<https://dev.mysql.com/doc/refman/8.4/en/multiple-column-indexes.html>
7. MySQL Reference Manual: How MySQL Uses Indexes 与 Index Condition Pushdown。
<https://dev.mysql.com/doc/refman/8.4/en/mysql-indexes.html>
8. Redis Documentation: Distributed Locks with Redis。
<https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>
9. Redis Documentation: Lua Scripting。
<https://redis.io/docs/latest/develop/programmability/eval-intro/>
10. Apache Kafka Documentation: Message Delivery Semantics 与 Producer Configuration。
<https://kafka.apache.org/documentation/>
11. RabbitMQ Documentation: Publisher Confirms、Consumer Acknowledgements 与 Quorum Queues。 <https://www.rabbitmq.com/docs/confirms>
12. Martin Kleppmann, *Designing Data-Intensive Applications*: 复制、一致性、分布式锁与消息语义的通用分析框架。

18.6 最后的复习建议

不要按章节逐字背诵。建议每题进行三轮训练：

1. 用 30 秒说结论。
2. 用 3 分钟展开核心机制和边界。
3. 接受 5 分钟追问，给出生产实例、故障路径和取舍。

当你能在没有固定措辞的情况下，根据面试官追问自由切换“原理、代码、故障和架构”四个层次，这套题才真正转化为高级开发能力。